

Robot Reinforcement Learning

What embodiment demands of RL

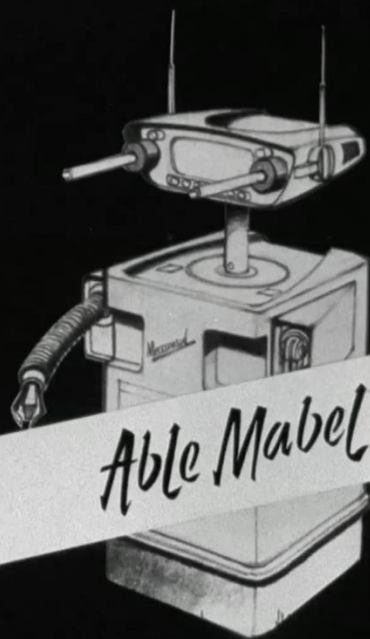
Georgia Chalvatzaki

RL Summer School 2026 · Milan



| Leisure time begin in 1976...

BBC



It is 1966.

The BBC promises that by 1976 a home robot will hand you your leisure time.

Able Mabel is that robot.

And she is **fully teleoperated**.

Hold onto this. Sixty years later, the best self-improving VLA still leans on a human teleoperator to close its loop. Autonomy, not control, is the part that never got solved.

What this tutorial argues

one thesis, three acts

Imitation and scale get you breadth. **Structure and RL** get you the rest — and the loop is **never done**.

Act I

Foundations

what fifteen years of robot RL actually established — and the sample budget that shaped all of it

Act II

What embodiment demands

five robotics primitives your algorithms assume but never teach: action, structure, reward, exploration, contact

Act III

The loop never closes

why continual RL for VLAs — reward without a verifier, improving without forgetting — is the open frontier

You already know RL. This tutorial is about everything the benchmarks left out — and the one problem nobody has closed.



Act I

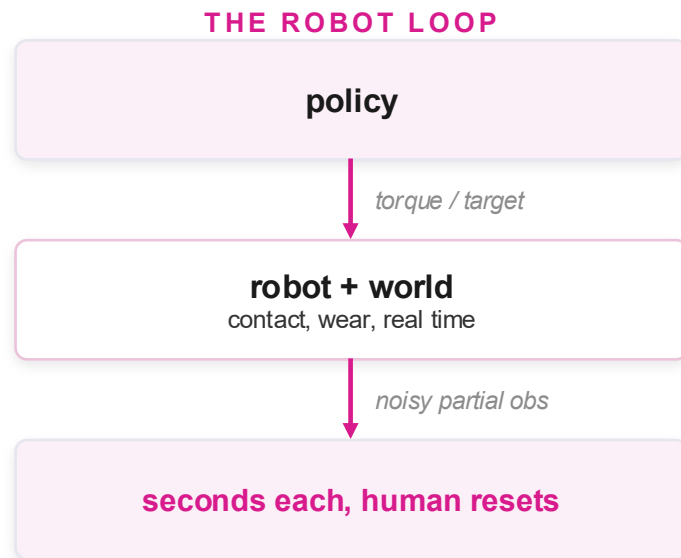
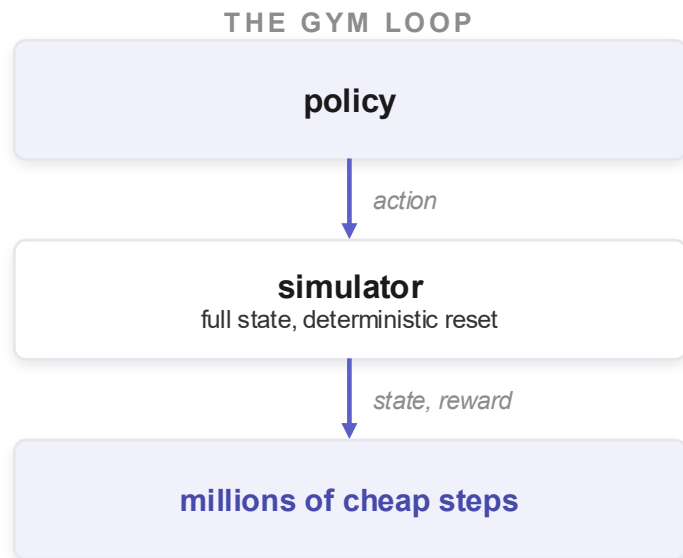
Foundations

why a robot is not a gym, and what fifteen years of robot RL actually established



Robots are not gym environments

the same MDP, but every assumption you rely on changes



Same equations. What breaks is everything around them: the state is hidden, samples are expensive, the reset is a person, and a bad action damages hardware.

The objective you know, re-read

the same return, but with a robotic perspective

$$\max_{\pi} \mathbb{E}_{\pi} \left[\sum_t \gamma^t r(s_t, a_t) \right]$$

the discounted return you have maximized a hundred times

s images, proprioception, force-torque: partial and delayed.

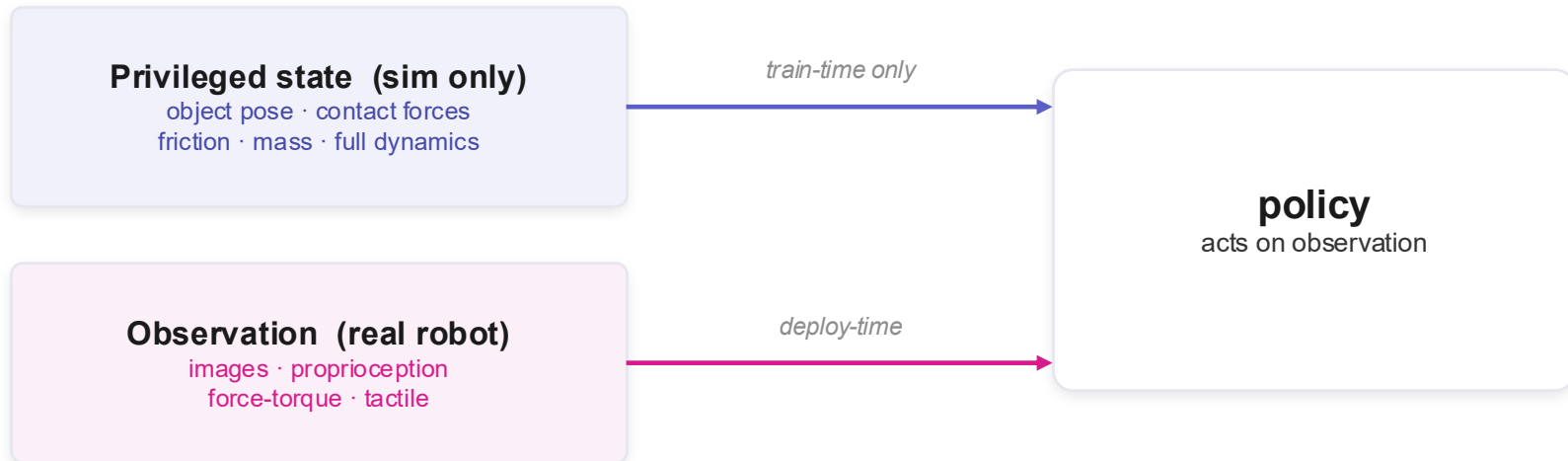
a a command to a controller, at some level of a hierarchy.

r you design it, and on contact tasks that is most of the work.

transitions are physics. You cannot reset them for free, and you cannot afford many.

What the policy can see, and what it can't

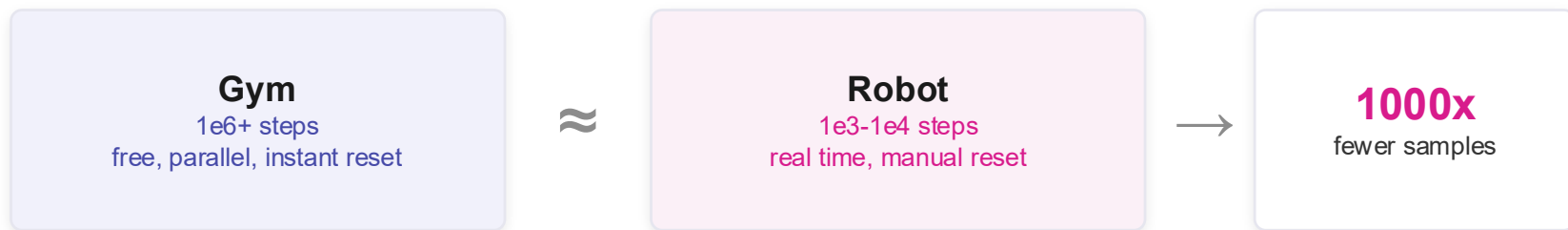
the gap that shapes every method that follows



The move. let the critic or a teacher use privileged state in sim, while the deployed policy reads the same intent from sensors alone. Asymmetric actor-critic, teacher-student, and sim-to-real all exploit this one asymmetry.

Why exploration hurts on hardware

the sample that is free in sim, costs you seconds and a human

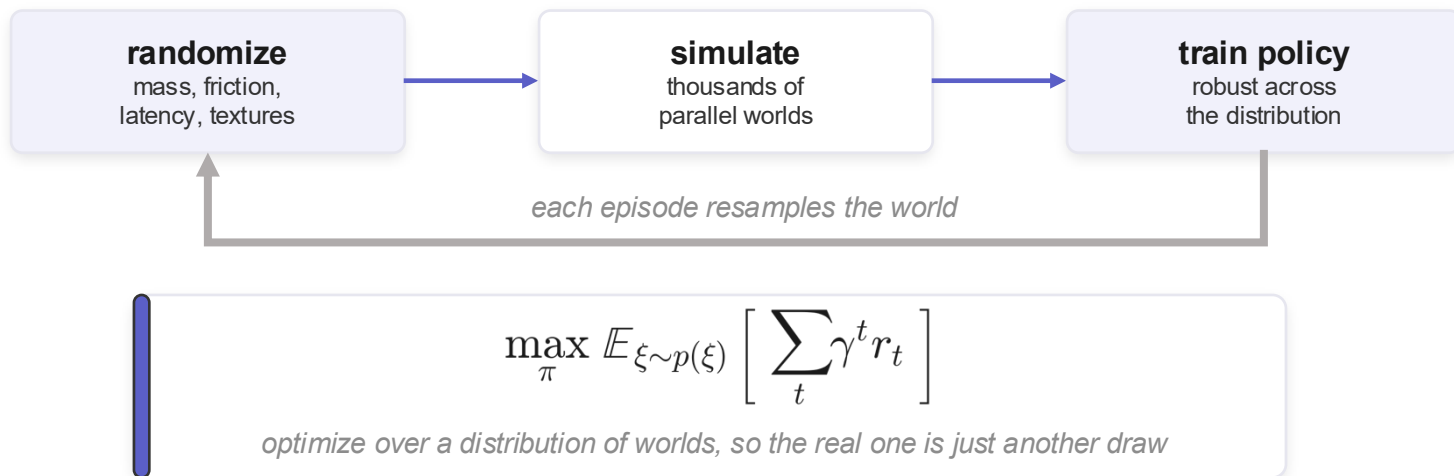


Consequence. robot RL is the search for what you can learn in sim, what you can pretrain, and what structure removes the need to learn at all.

Read the rest of Act I as three answers to this one constraint: simulation, structure, and reuse.

Sim-to-real: make reality one more sample

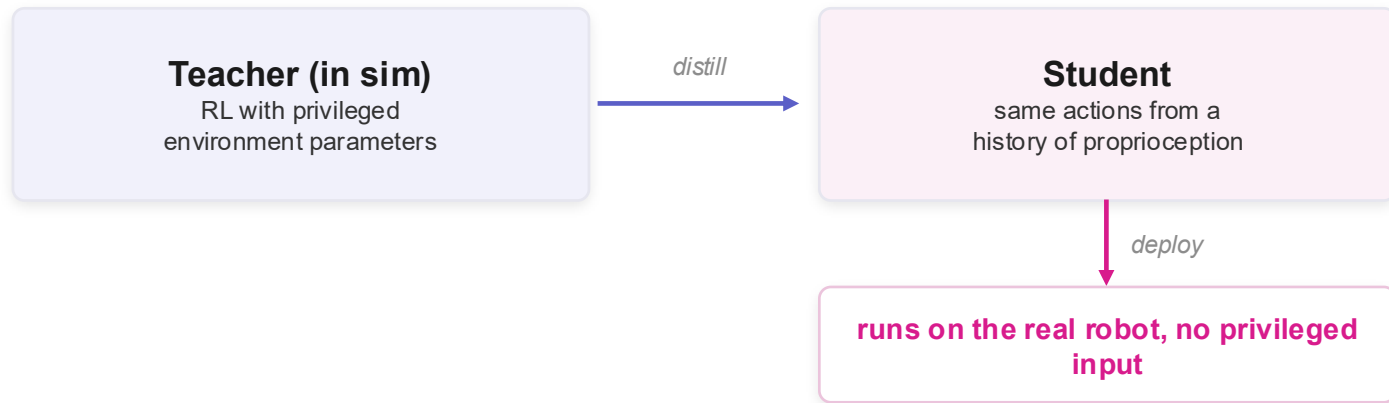
domain randomization, the mechanism behind zero-shot transfer



Why it works. the model never has to be correct, only wide enough that reality lands inside its support.

Training with information you won't have

the teacher-student pattern, in one diagram



$$\min \| a_{\text{student}}(o_{1:t}) - a_{\text{teacher}}(s^{\text{priv}}) \|$$

The trick. the student can't see friction or terrain, so it **infers them online** from how the robot has been moving.

Fifteen years in one line

the trajectory the milestones trace



structure and simulation early; scale in the middle; robustness and efficiency now

ERA 1 · STRUCTURE AND MODELS

Classical robot RL, before deep nets

structure and models bought sample efficiency early

Policy search. optimize a few parameters of a structured motor primitive, not a big network.

Model-based. PILCO learns a probabilistic dynamics model, solving cart-pole in a handful of trials.

The survey. Kober, Bagnell & Peters name the real obstacles: data cost, reward, priors.

few params + a model $\Rightarrow$ few real trials

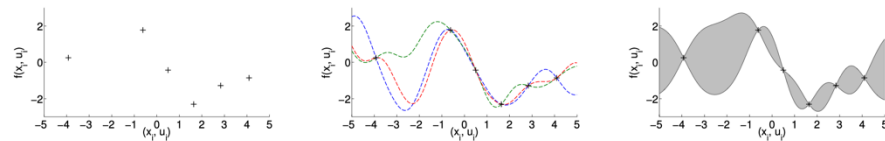


Figure 1. Small data set of observed transitions (left), multiple plausible deterministic function approximators (center), probabilistic function approximator (right). The probabilistic approximator models uncertainty about the latent function.

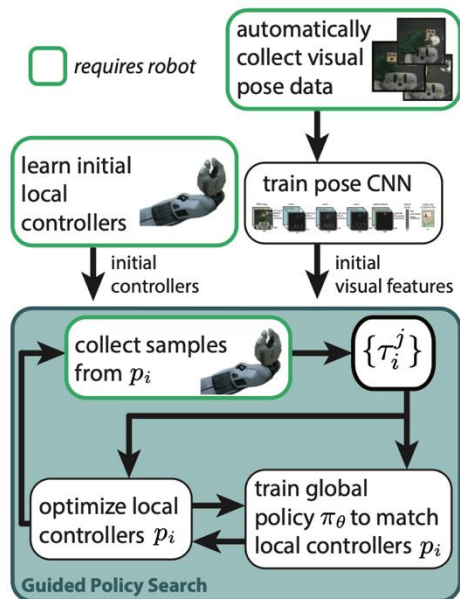
ERA 2 · THE DEEP RL TURN

Deep visuomotor policies: pixels to torques

end-to-end control learned on a real manipulator (GPS)

Idea. guided policy search turns a hard RL problem into supervised learning under a trajectory-optimization teacher.

Why it mattered. the first convincing end-to-end perception-to-control on real hardware.



$\pi_\theta : \text{image} \rightarrow \text{motor torques}$

Hindsight Experience Replay: learn from failure

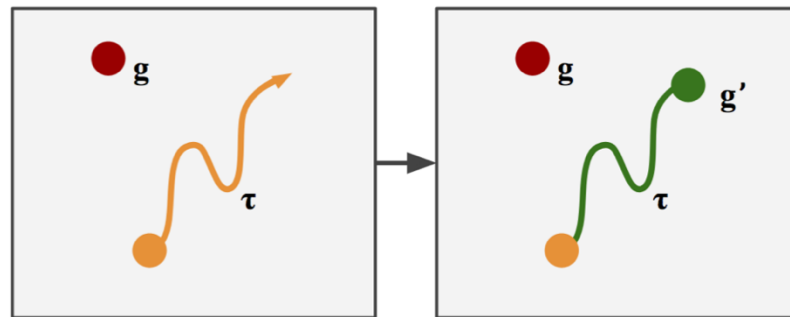
turn every miss into a labeled success

Idea. if you aimed for goal g and reached g' , relabel the episode as a success for g' .

Why it mattered. made sparse-reward, goal-conditioned manipulation trainable without reward shaping.

Goal-conditioned RL

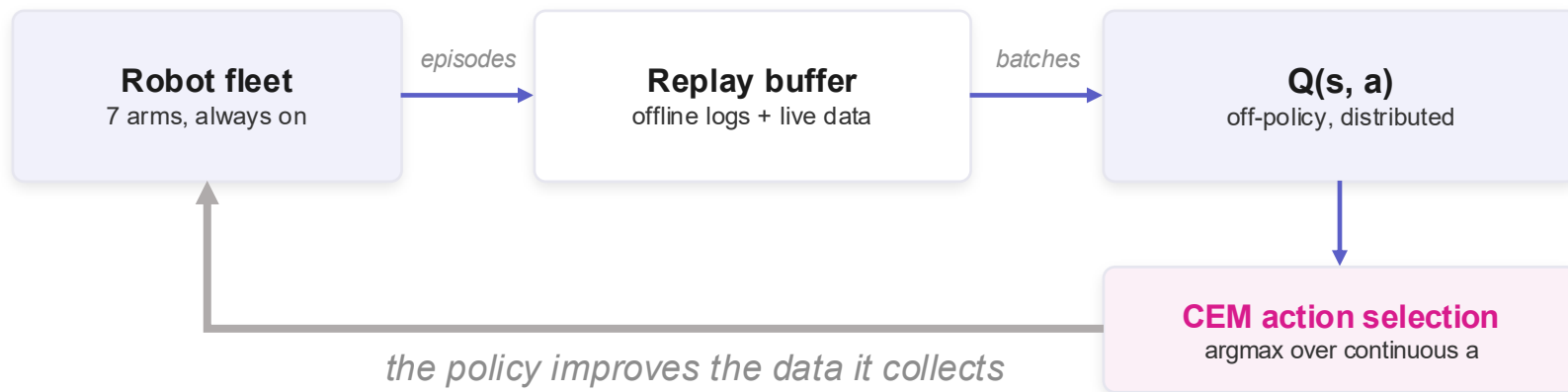
$$\tau \sim \pi(a | s, g)$$



$g \rightarrow g'$: a failure for g is a success for g'

Grasping as value learning at scale

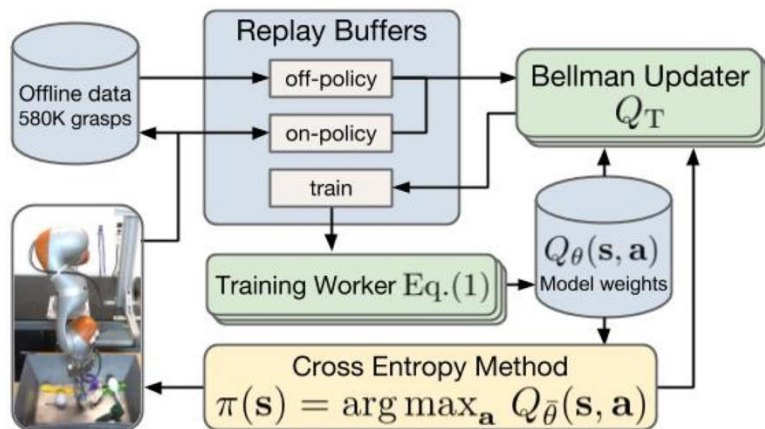
QT-Opt: no policy network, just a Q-function and a fleet



$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a') \quad a^* = \arg \max_a Q(s, a)$$

96% grasp success from raw pixels

vision-based grasping across a fleet, generalizing to new objects



96% on unseen objects **580k** real grasp attempts

Soft Actor-Critic on real robots

maximum-entropy RL, stable enough to train outside a sim

Idea. reward return and policy entropy together; the policy stays exploratory and robust to hyperparameters.

Why it mattered. learned a Minitaur gait directly in the real world; the default modern off-policy baseline.

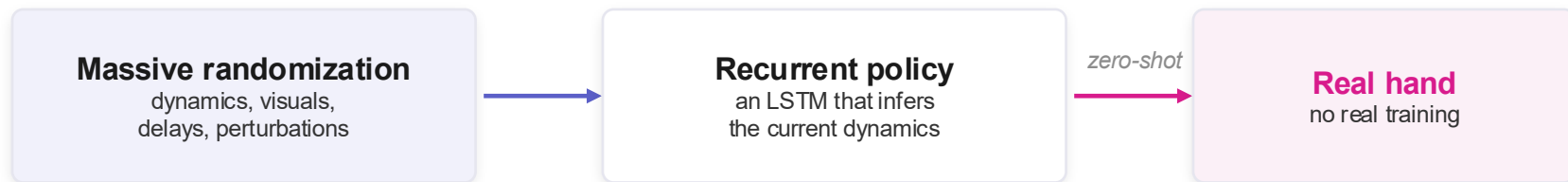
$$\max_{\pi} \mathbb{E} \left[\sum_t r_t + \alpha \mathcal{H}(\pi(\cdot | s_t)) \right]$$



ERA 3 · DEXTERITY AND LEGS, VIA SIM-TO-REAL

Dexterity from a randomized simulation

OpenAI's hand: solve everything in sim, transfer zero-shot



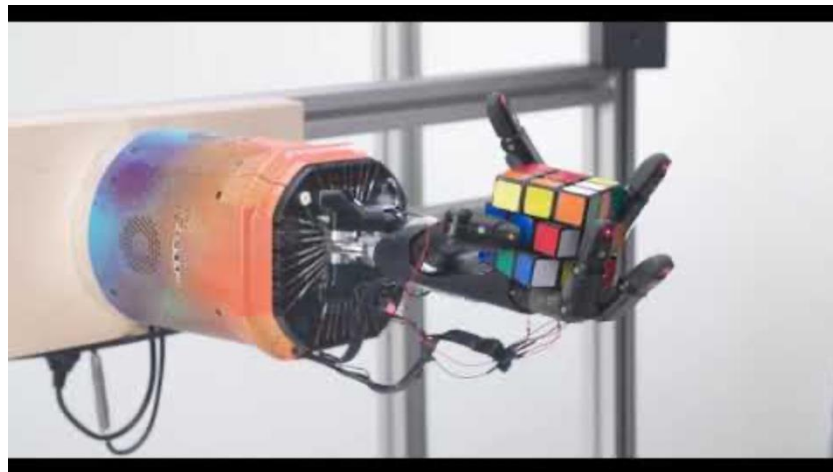
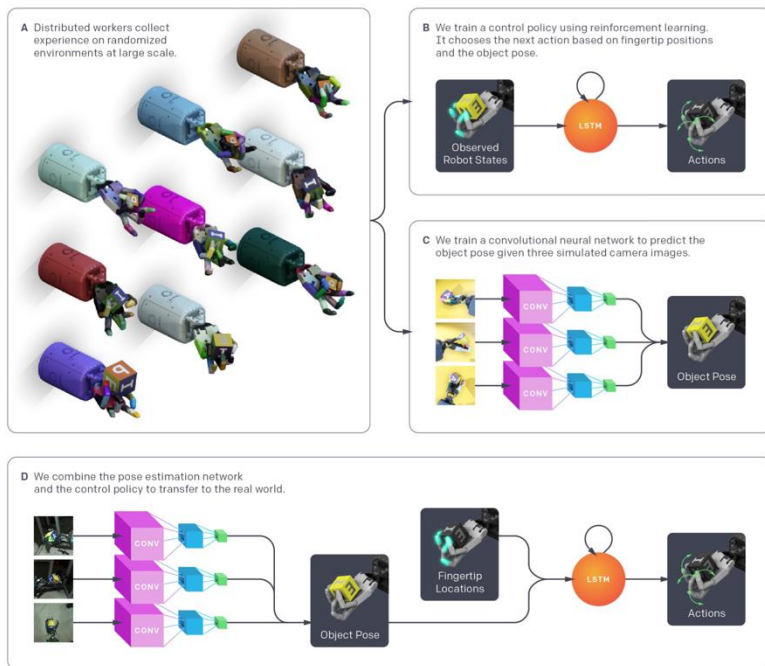
$$\max_{\pi} \mathbb{E}_{\xi \sim p(\xi)} \left[\sum_t \gamma^t r_t \right]$$

the policy reads the dynamics from its own history; the memory infers them

The bet. if the distribution of sims is wide enough, the real hand is just a hard sample inside it.

A five-fingered hand solving a Rubik's cube

trained only in simulation, never on the real robot



0 real-world training episodes 100% in simulation, transferred

ANYmal: learn the actuator, transfer the gait

model the dynamics you can't randomize away, then deploy



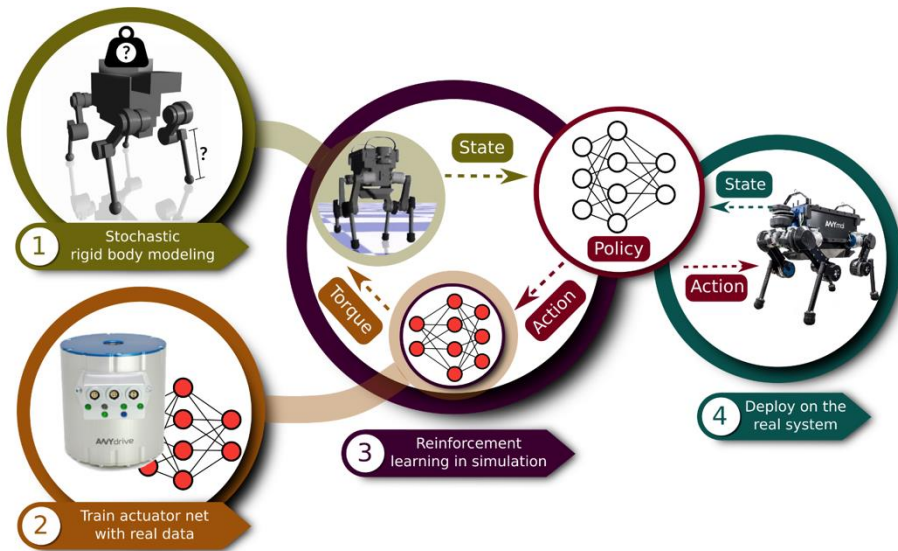
$$\hat{\tau}_t \approx f(q_{t-H:t}, q_{t-H:t}^{\text{des}})$$

the one part of the dynamics you fit from real data instead of randomizing

The lesson. randomize what you don't know; learn the part you can measure. The rest transfers.

Agile gaits and fall recovery on a quadruped

dynamic walking, running, and getting back up after a fall



RL beat hand-crafted controllers

real hardware, zero-shot

Hwangbo et al., Learning agile and dynamic motor skills for legged robots, Science Robotics 2019, Miki et al., Learning robust perceptive locomotion for quadrupedal robots in the wild, Science Robotics 2022

RMA: rapid online adaptation

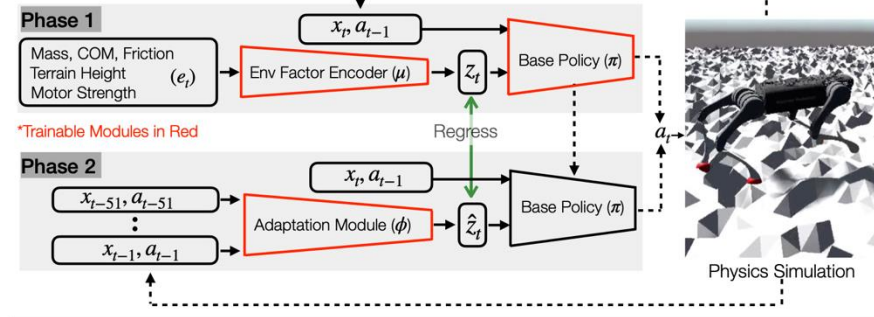
the teacher-student pattern, made fast on hardware

Idea. a base policy plus a small module that infers environment 'extrinsics' from recent proprioception, in milliseconds.

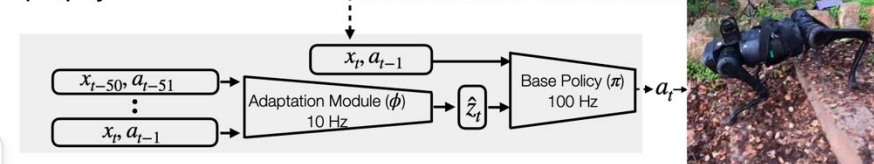
Why it mattered. robust legged locomotion without explicit system identification, adapting as the ground changes.

$$\hat{z} = \phi(o_{t-H:t}), \quad a = \pi(o_t, \hat{z})$$

A) Training in Simulation



B) Deployment

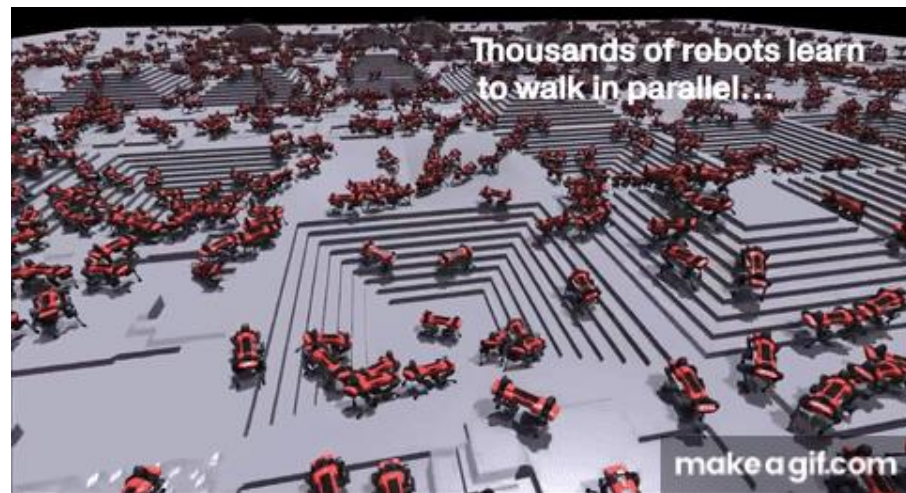


Learning to walk in minutes

massively parallel simulation collapses training time

Idea. thousands of robots in one GPU simulator (Isaac Gym) gather experience in parallel.

Why it mattered. minutes, not days; GPU-parallel sim became the default substrate for robot RL.



$$1 \text{ GPU} \times N_{\text{robots}} \Rightarrow \text{minutes}$$

Learning directly on the real robot, efficiently

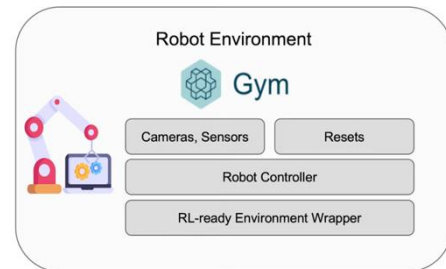
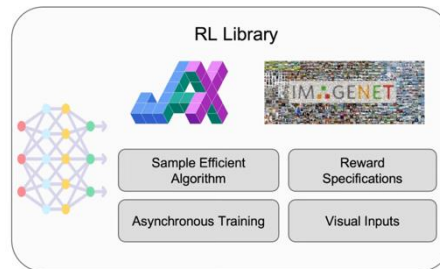
modern off-policy RL that trains on hardware in minutes

Walk in the Park. a quadruped learns to walk in about 20 minutes, no simulator.

SERL. a reusable system makes real-world RL for contact-rich manipulation repeatable.



no simulator : ≈ 20 min on hardware

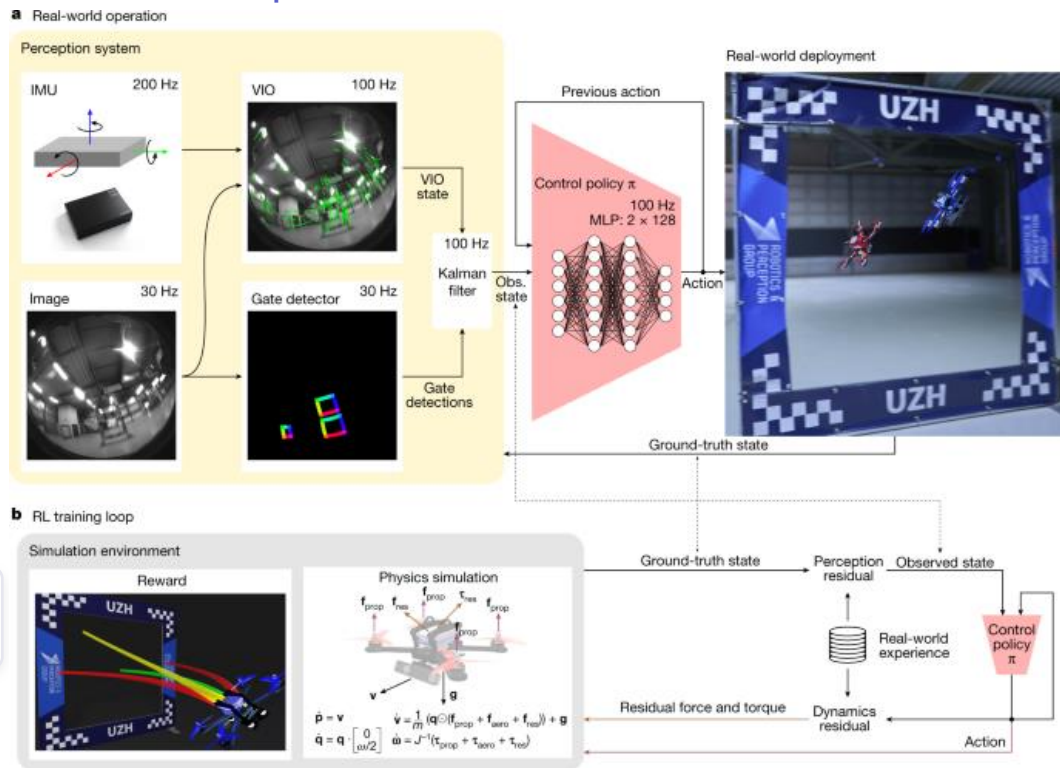


Champion-level drone racing

RL from onboard sensing beats human world champions

Idea. an RL policy pushes a physical quadrotor to its dynamic limits, flying from onboard perception.

Why it mattered. first superhuman performance against human champions in a physical sport.



$o_{\text{onboard}} \rightarrow a$ at the dynamic limit

Champion-level drone racing

RL from onboard sensing beats human world champions

nature

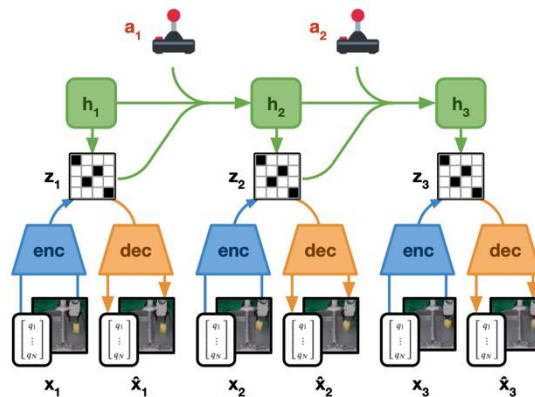


World models on physical robots

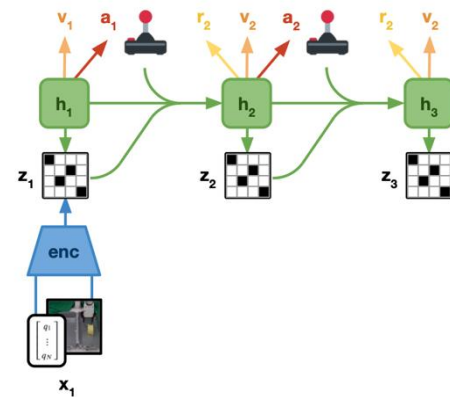
learn a latent dynamics model, train the policy in imagination

Idea. DayDreamer learns a latent model from real interaction, then improves the policy mostly inside it.

Why it mattered. real-robot tasks learned directly on hardware in hours, by dreaming the rollouts.



(a) World Model Learning



(b) Behavior Learning

$$\hat{P}_{\psi}(s' | s, a), \quad \pi \text{ trained in imagination}$$

What fifteen years taught us

three levers, used in every success above

Structure

primitives, models, symmetry,
priors: shrink what must be
learned

Simulation

parallelism + randomization:
learn where data is free,
transfer to real

Reuse

offline data, demonstrations,
pretraining: don't relearn what
you have

every milestone is one or more of these, applied to the sample-budget problem

| What the milestones don't teach

they show what works; they assume the robotics you still need

The gap. every result above quietly made choices about action spaces, rewards, sensing, and structure. Those choices are the robotics.

Act II. five primitives RL theory does not give you: action representation, structure, reward and data, exploration, and sensing contact.

Each is a concept with a diagram, then a system that makes it concrete on real hardware.

Act II

What embodiment demands

five robotics primitives your algorithms assume but never teach you



Five primitives RL theory leaves out

the robotics your algorithms quietly run on

- 1 **Action representation** where the policy acts, and what the controller does
- 2 **Structure** symmetry and priors as a substitute for data
- 3 **Reward and data** designing reward, and generating data at scale
- 4 **Exploration** multimodal behavior when one mode is not enough
- 5 **Sensing contact** observation as a design choice, beyond vision

| One way to organize them

five layers, from what the policy represents to how the body moves

Object representations	pose, geometry, features the policy can act on
Uncertainty representations	beliefs and what the robot does not know
Interaction representations	affordances and contact, where robot meets world
Structured decision-making	priors, symmetry, hierarchy in the policy itself
Whole-body coordination	many joints and contacts acting as one

a map, not a hierarchy of importance: the five primitives cut across these layers

1

PRIMITIVE 1 OF 5

Action representation and control hierarchy

the action space is a design choice, not a given

| Where does the policy act?

how much of control you hand to a controller you already trust

Joint torques

policy controls everything; hardest to learn

The choice. where you split the work between policy and controller.

Joint position targets + PD

policy sets setpoints; a PD loop does the rest

Consequence. the same task is easy or hopeless depending on this one decision.

Task-space target + IK / OSC

policy thinks in end-effector space

RL habit. flat torque vectors, because the benchmark gave you that.

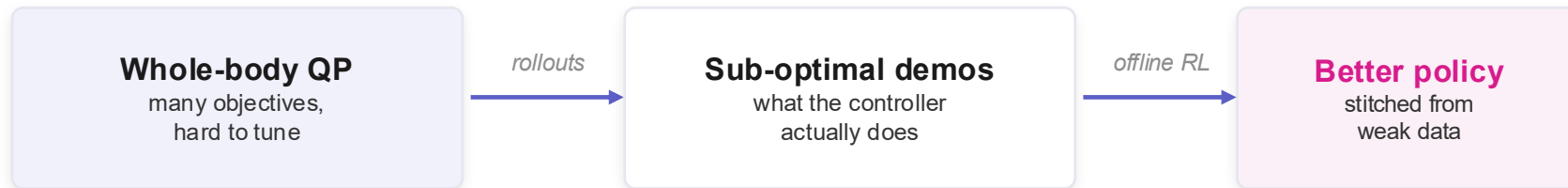
Setpoint to a whole-body QP

controller resolves redundancy + constraints

top: the policy does more · bottom: the controller does more

Don't replace the controller. Correct it.

a weak whole-body controller is still a strong prior

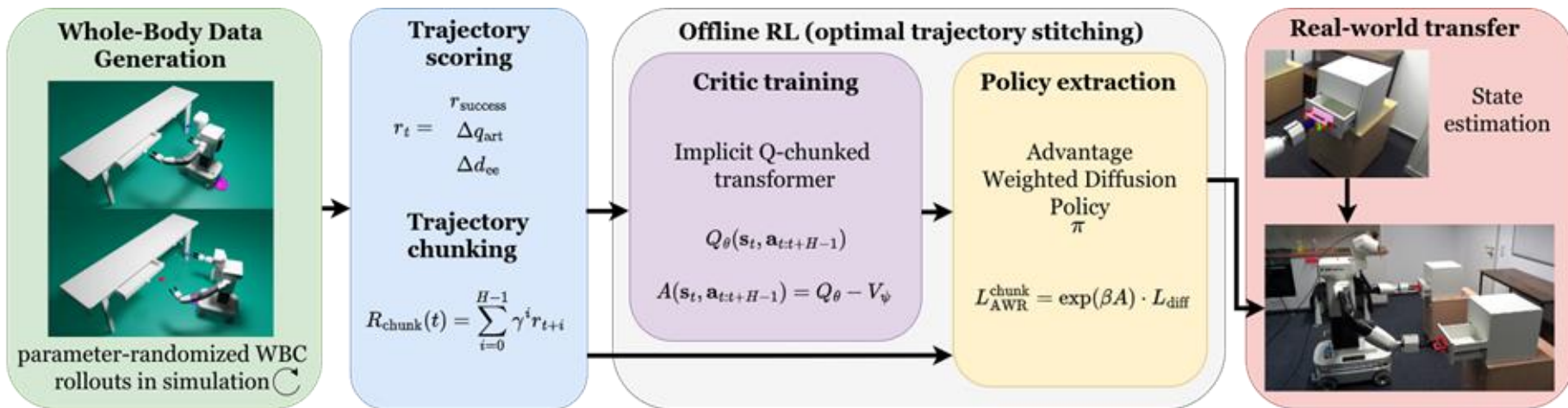


$$\min_{\ddot{q}} \sum_i w_i \left\| J_i \ddot{q} - \ddot{x}_i^{\text{des}} \right\|^2 \quad \text{s. t. limits, contacts}$$

The reframe. treat the controller as a source of imperfect but physically valid trajectories, exactly what offline RL needs, and let RL improve only the policy.

Offline-RL for Whole-body MoMa

a weak whole-body controller is still a strong prior



- Generate **diverse whole-body demos** with a randomized WBC
- Label with rewards, then train a **Q-chunked transformer critic via IQL + AWR Diffusion Policy**
- **Zero-shot deploy** on the real robot

Results in whole-body articulated manipulation: Cupboard

WBC



BC (DP)



IDQL [45]



WHOLE-MoMa



Where the gain comes from

the learned policy against the controller and imitation baselines

TABLE III: Simulation results for WHOLE-MoMa and all baselines. Success rates are evaluated over 50 episodes per task, reported with 95% confidence intervals. Time to success is averaged over successful trials.

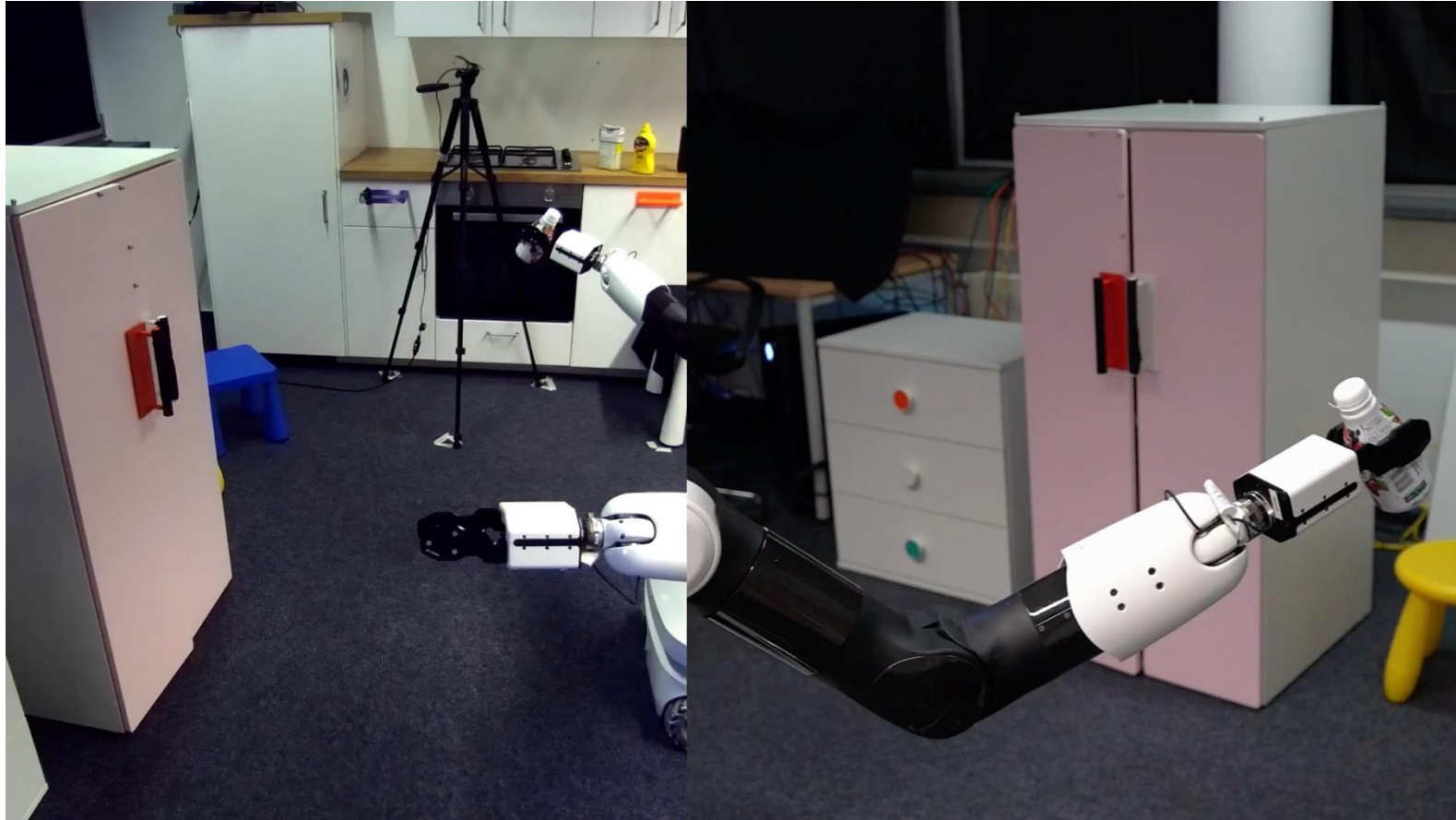
Metric / Method	WBC Policy	BC (Diffusion Policy [11])	RL (TD3 [54]) @ 200k	IQL+DDPG_BC [42]	IDQL [45]	RISE [46]	WHOLE-MoMa
Door task:							
Success %	86 [73.8, 93.0]	78 [64.8, 87.2]	88 [76.2, 94.4]	86 [73.8, 93.0]	90 [78.6, 95.7]	92 [81.2, 96.8]	98 [89.5, 99.6]
Time to success (s)	7.8	11.3	10.9	11.0	11.0	10.9	10.6
Drawer task:							
Success %	68 [54.2, 79.2]	70 [56.2, 80.9]	44 [31.2, 57.7]	64 [50.1, 75.9]	72 [58.3, 82.5]	70 [56.2, 80.9]	80 [67.0, 88.8]
Time to success (s)	14.4	15.0	19.3	20.5	18.7	18.5	17.4
Cupboard task:							
Success %	52 [38.5, 65.2]	48 [34.8, 61.5]	0 [0.0, 7.1]	6 [2.1, 16.2]	64 [50.1, 75.9]	64 [50.1, 75.9]	78 [64.8, 87.2]
Partial success %	80 [67.0, 88.8]	78 [64.8, 87.2]	42 [29.4, 55.8]	54 [40.4, 67.0]	88 [76.2, 94.4]	90 [78.6, 95.7]	100 [92.9, 100]
Time to success (s)	14.4	19.2	—	21.1	19.6	19.1	18.7

Read it as. behavior cloning copies the controller's mistakes; offline RL stitches a better policy from the same weak data.

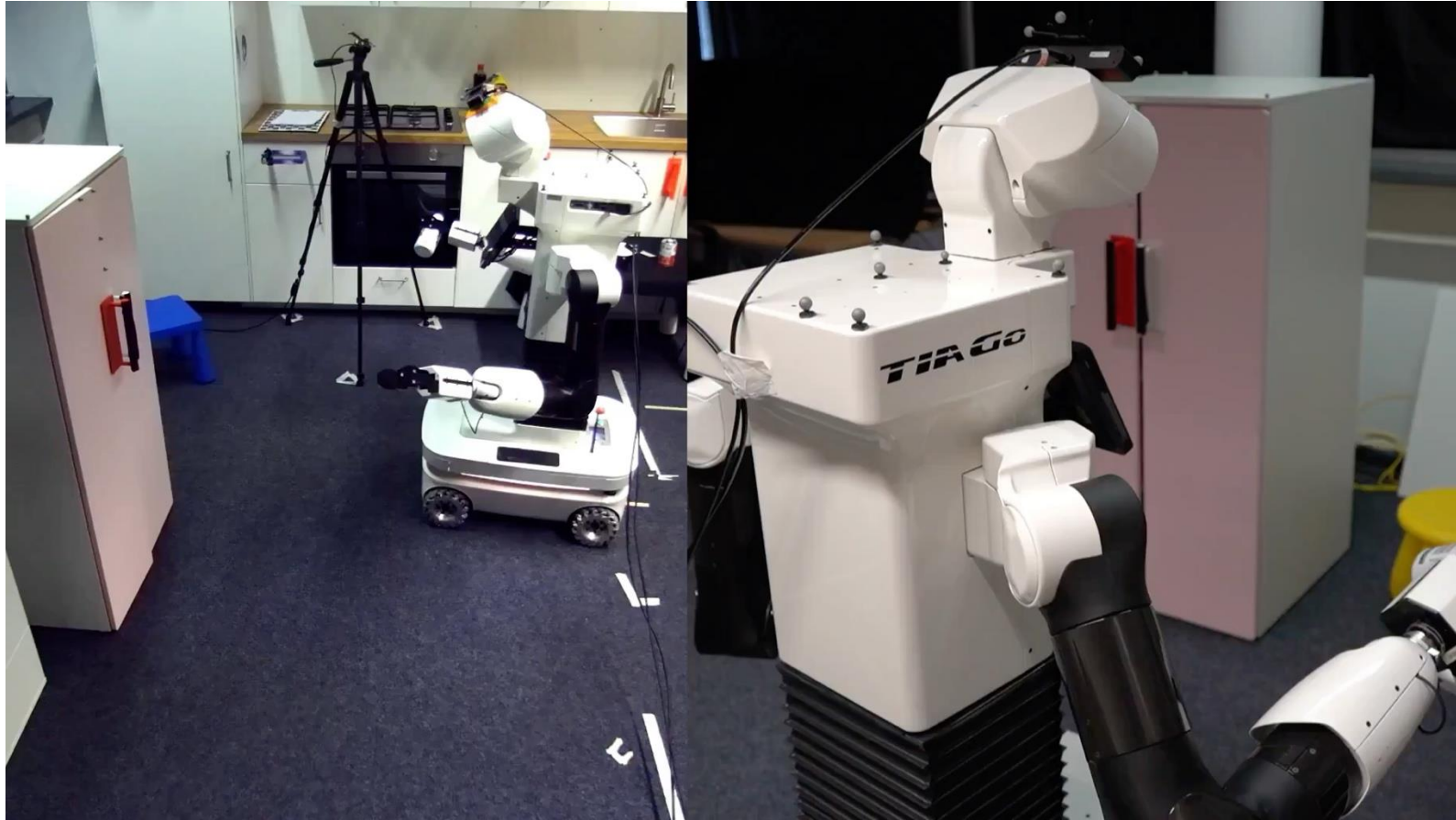
RealCupboardOpenAndPlace Task: WBC-based policy



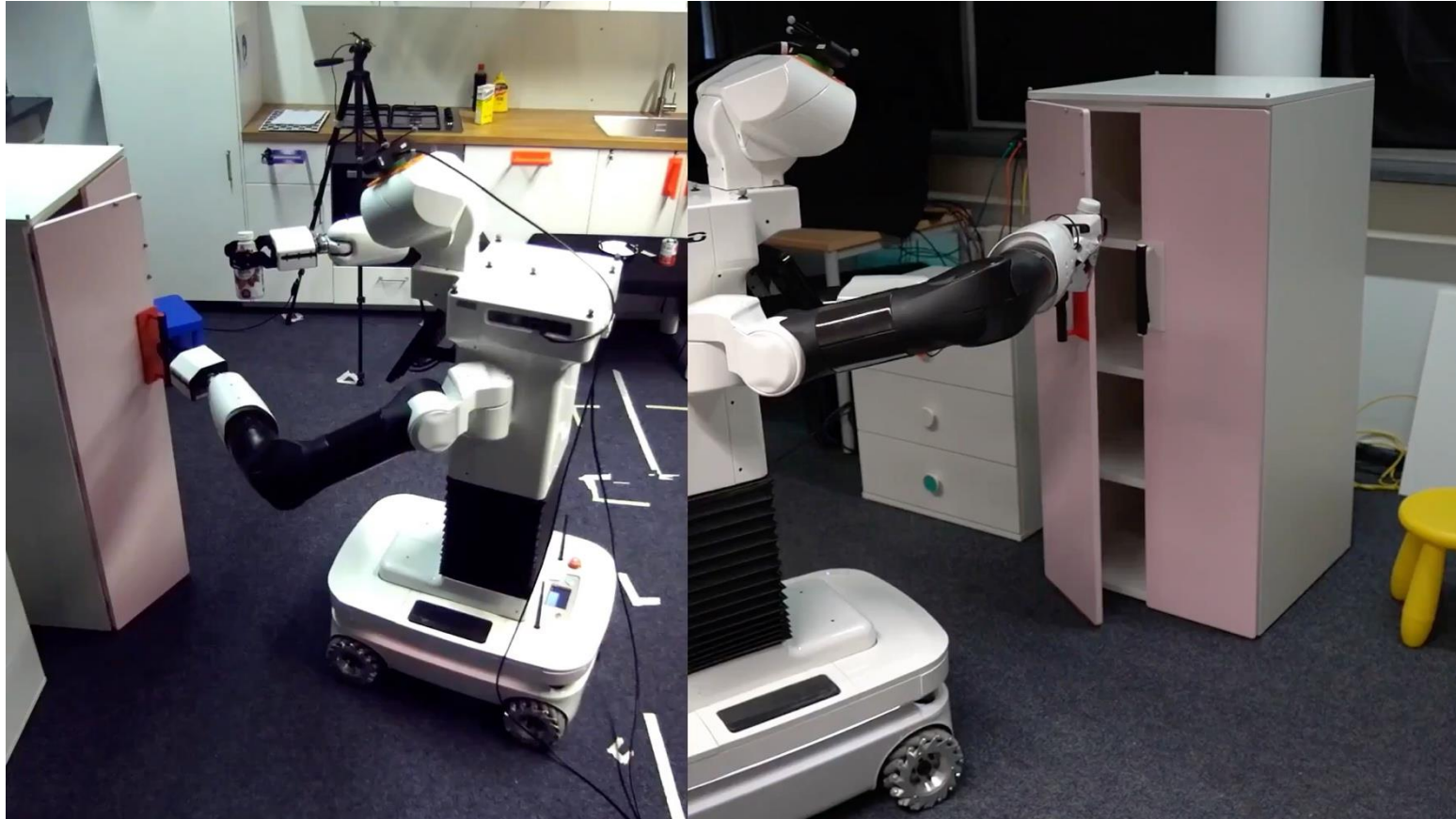
RealCupboardOpenAndPlace Task: Imitation Learning



RealCupboardOpenAndPlace Task: WHOLE-MoMa



RealCupboardOpenAndPlace Task: Robustness demo



Whole-body articulated manipulation, zero-shot to real



80%

bimanual drawer

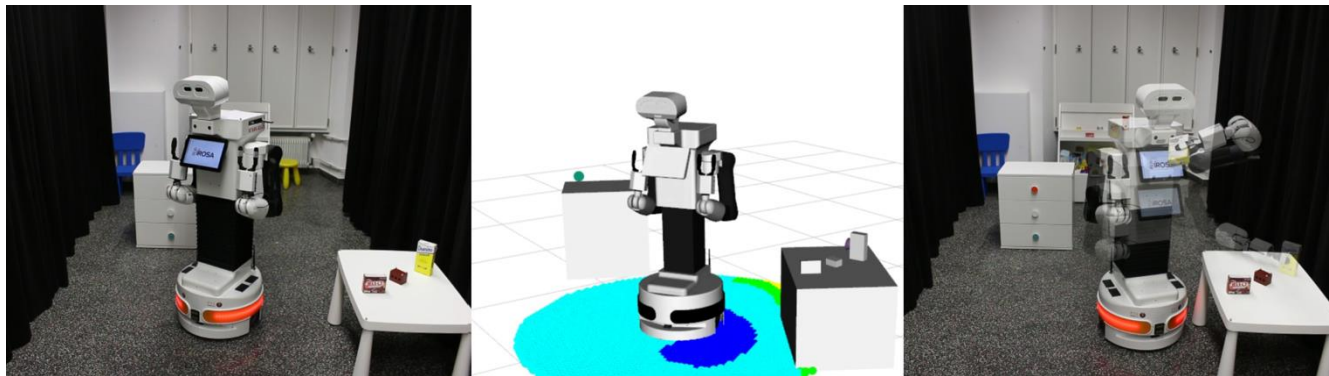
68%

cupboard open + place

no teleop, no real-world training data

Hybrid action spaces for mobile grasping

BHyRL: a reachability prior over a discrete-continuous action

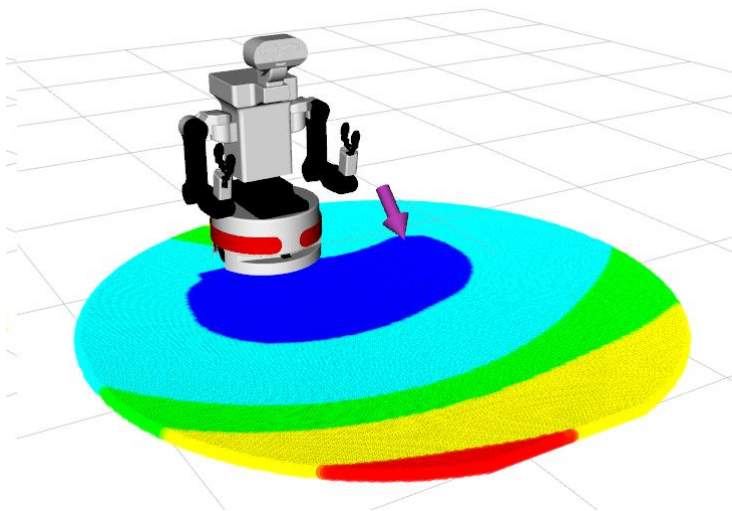


- **Hybrid action-space RL** discrete and continuous action decisions in Mobile Manipulation
- We learn a **reachability behavioral prior** for selecting kinematically reachable base poses
- New algorithm: **Boosted Hybrid RL** (BHyRL) for transferring knowledge from behavior priors

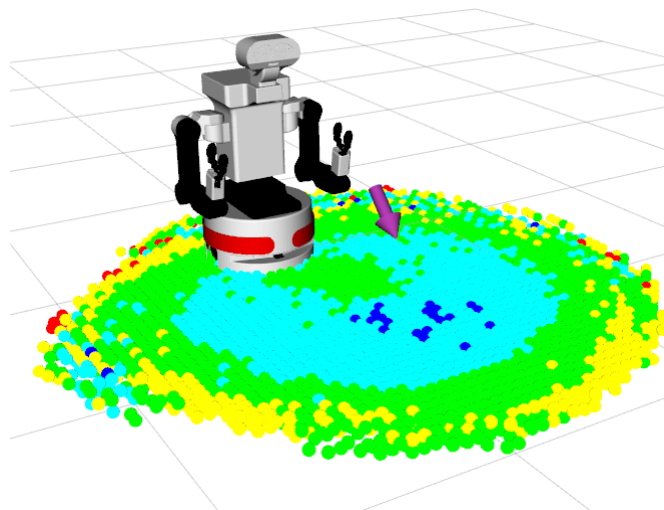
Structure behavior priors

Using robot reachability maps – embodiment specific

Q-reachability map



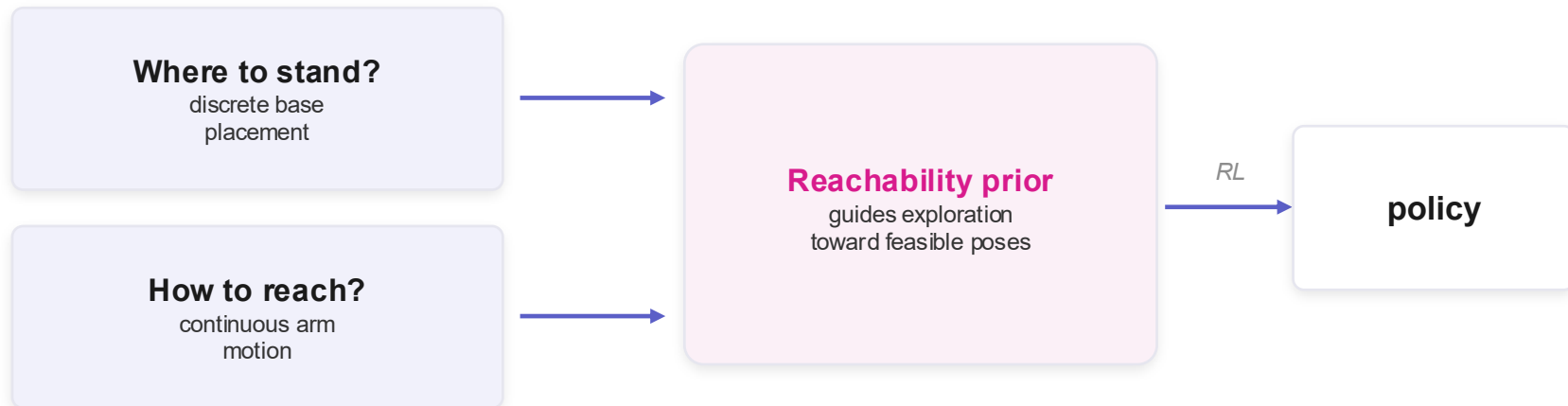
Inverse Map



We train an RL agent on a reachability dataset, we get a **smooth** Q-reachability map

Hybrid action spaces for mobile grasping

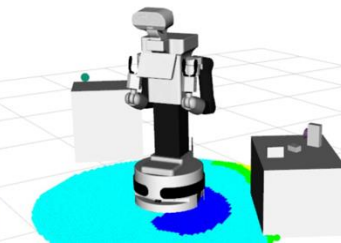
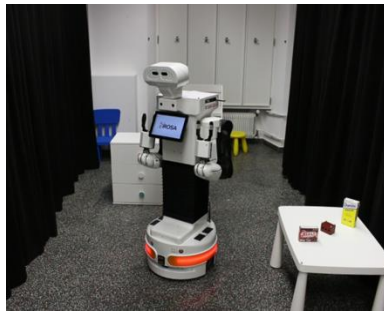
BHyRL: a reachability prior over a discrete-continuous action



$$a = (a_{\text{base}}, a_{\text{arm}}) \sim \pi(\cdot \mid s; \rho_{\text{reach}})$$

Mobile grasping as BHyRL

the prior makes a sparse, long-horizon task learnable



Handle changing
observation spaces
across tasks?

Critic update

$$\left. \begin{aligned} Q_{\phi_0}^0 &= \rho_{\phi_0}^0 \\ Q_{\phi_1}^1 &= \rho_{\phi_0}^0 + \rho_{\phi_1}^1 \\ &\vdots \\ Q_{\phi_T}^T &= \rho^0 + \rho^1 + \dots + \rho_{\phi_T}^T \end{aligned} \right\} Q_{\phi_T}^T = \sum_{i=0}^{T-1} \rho^i + \rho_{\phi_T}^T$$

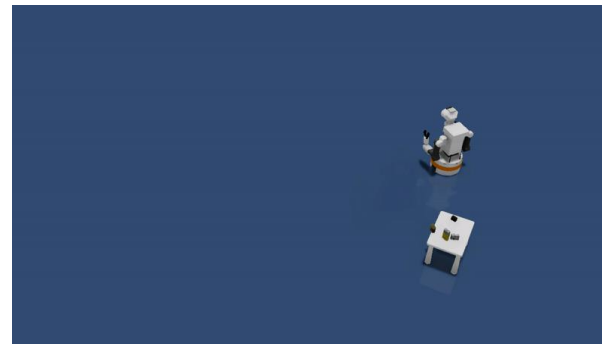
$$\mathcal{L}(\phi^{Q^T}) = \mathbb{E}_{\substack{s, a, s', r \\ \sim D^T}} \left[(Q_{\phi}^T(s, a) - y^T)^2 \right]$$

Actor update

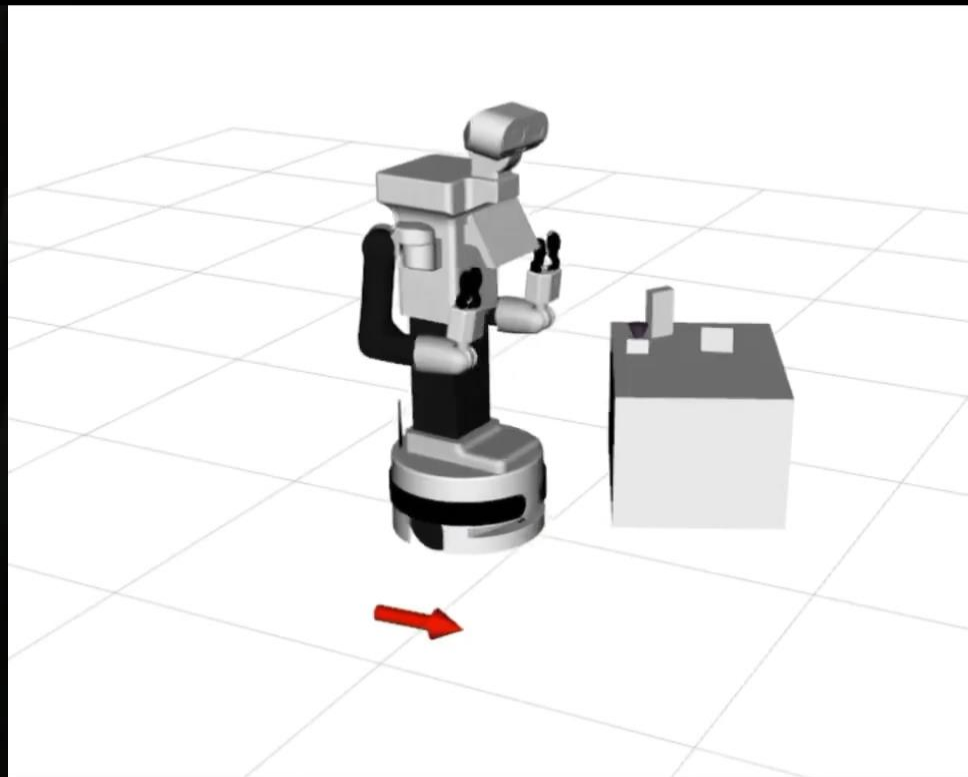
$$\mathcal{L}(\theta^{\pi^T}) = \mathbb{E}_{\substack{s \sim D^T, \\ a \sim \pi_{\theta}^T(\cdot|s)}} \left[Q^T(s, a) \right]$$

$$- \alpha \text{KL}(\pi^{T-1}(\cdot|s^{T-1}) || \pi_{\theta}^T(\cdot|s))$$

Mobile grasping as BHyRL



On the right, we visualize the learnt Q function of the agent



2

PRIMITIVE 2 OF 5

Structure as a substitute for data

inductive bias is sample efficiency you do not have to pay for

Symmetry as inductive bias

if the task has a symmetry, bake it into the network



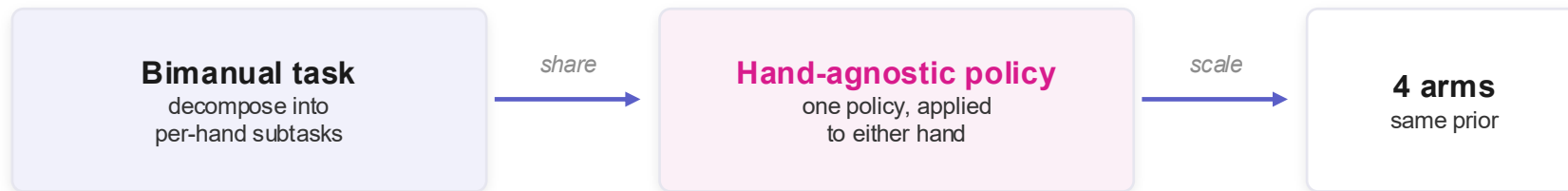
$$\pi(g \cdot s) = g \cdot \pi(s) \quad \forall g \in G$$

mirror the scene and the policy mirrors its action, by construction

Payoff. every left-hand interaction is automatically a right-hand one. The symmetry is exact, never learned, so the data you have counts twice.

| One policy for two hands, by symmetry

SYMDEX: equivariant per-hand policies for bimanual manipulation

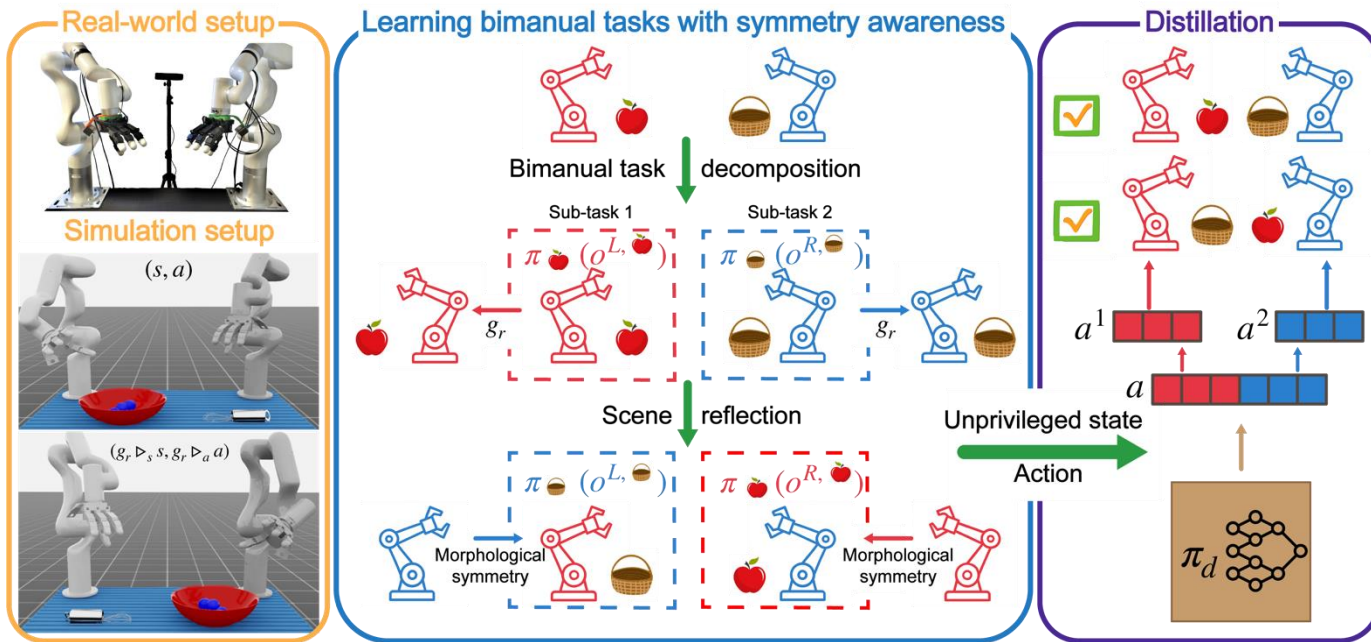


Experience sharing. by construction, one arm's interaction trains the other.

Where it wins. the margin over baselines grows most on asymmetric-role tasks, exactly where coordination is hardest.

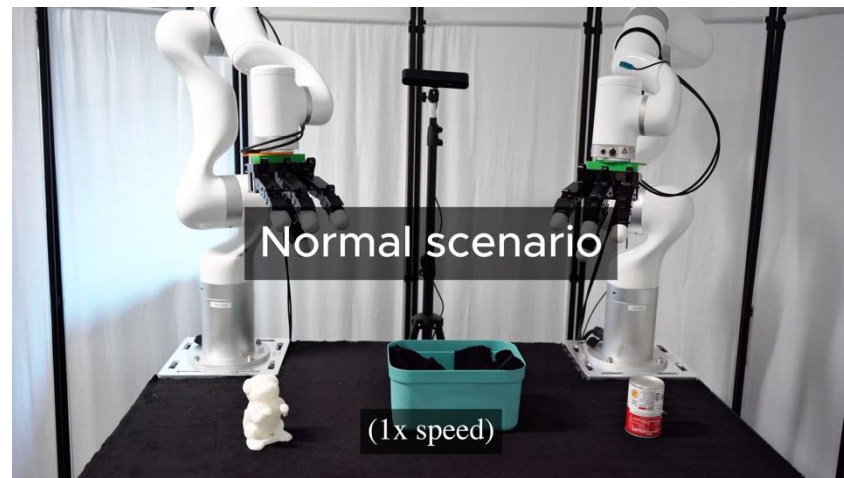
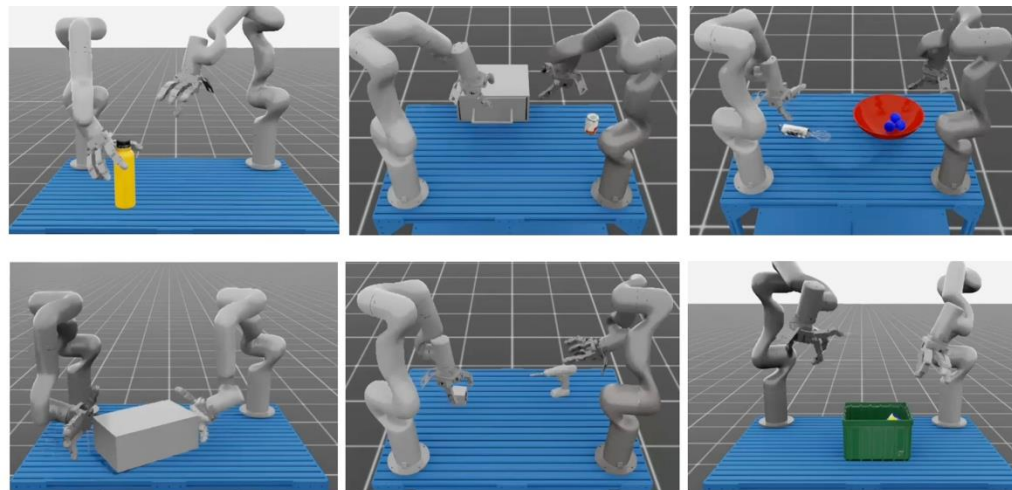
One policy for two hands, by symmetry

SYMDEX: equivariant per-hand policies for bimanual manipulation



Ambidextrous from one shared policy

the same policy drives either hand, and scales past two

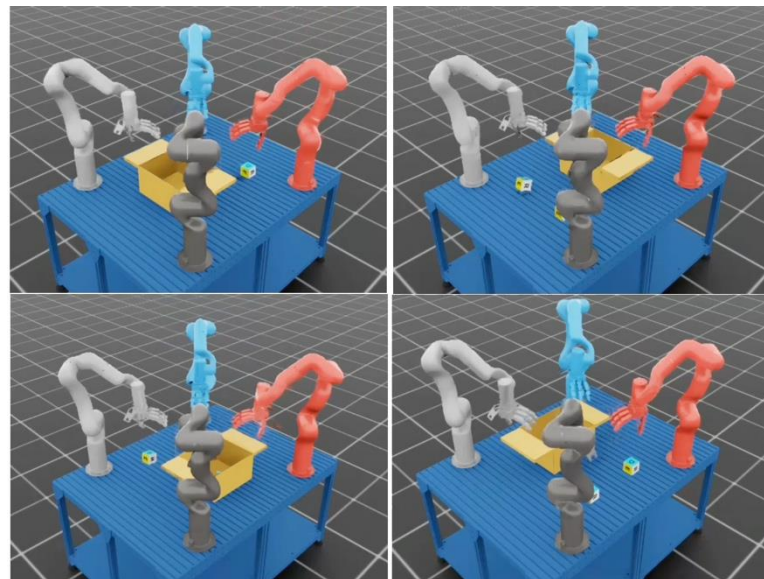
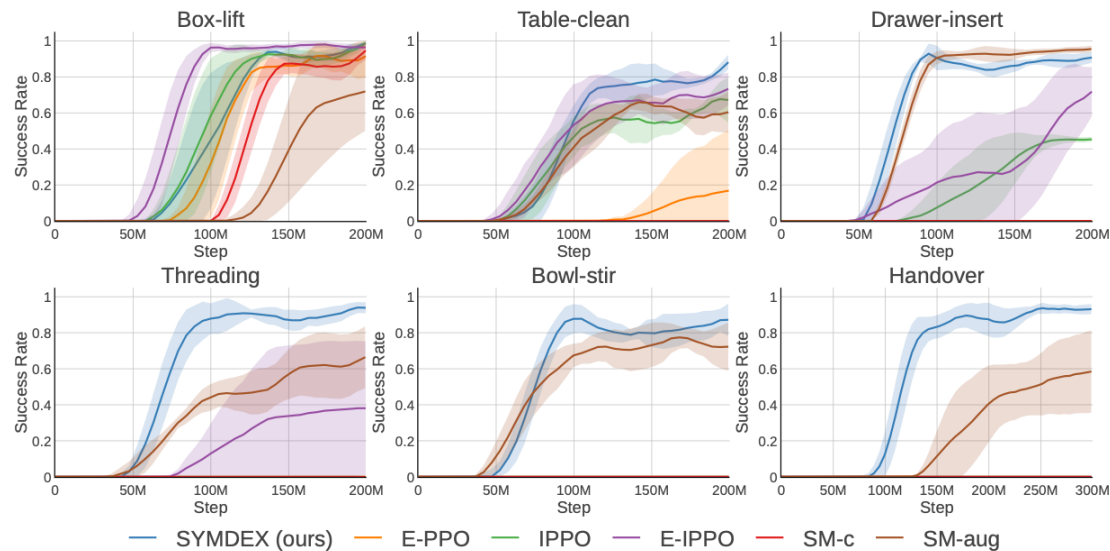


1 policy either hand

4 arms same symmetry prior

What symmetry buys, measured

the gain is largest exactly where the task is hardest



The takeaway. structure is not a tweak; it changes what is learnable inside a real sample budget.

3

PRIMITIVE 3 OF 5

Reward design and data at scale

the score function does not exist; you design it, then scale it

Rewarding a physical task

RL people reward games; robots need contact-aware shaping

$$r = w_1 r_{\text{align}} + w_2 r_{\text{contact}} + w_3 r_{\text{goal}} + w_4 r_{\text{reset}}$$

the same four terms compose across lifting, pick-place, and insertion

The trap. copying a reference trajectory is over-constrained and brittle; the robot learns the demo, not the task.

A better basis. reward the relationship (align, make contact, reach a flexible goal) and budget for resets. The terms transfer.

Cost. about an hour per task to specify, versus a day tuning trajectory rewards.

| The reset problem

in the real world, who puts the scene back?

Sim

reset is free
and instant

VS

Robot

every episode needs
the scene restored

Make it part of the task. include a reset term so the policy learns to undo its own mess; that is the reset term in the reward above.

Why it matters. reset-free behavior is what lets RL run unattended, the precondition for collecting data at scale.

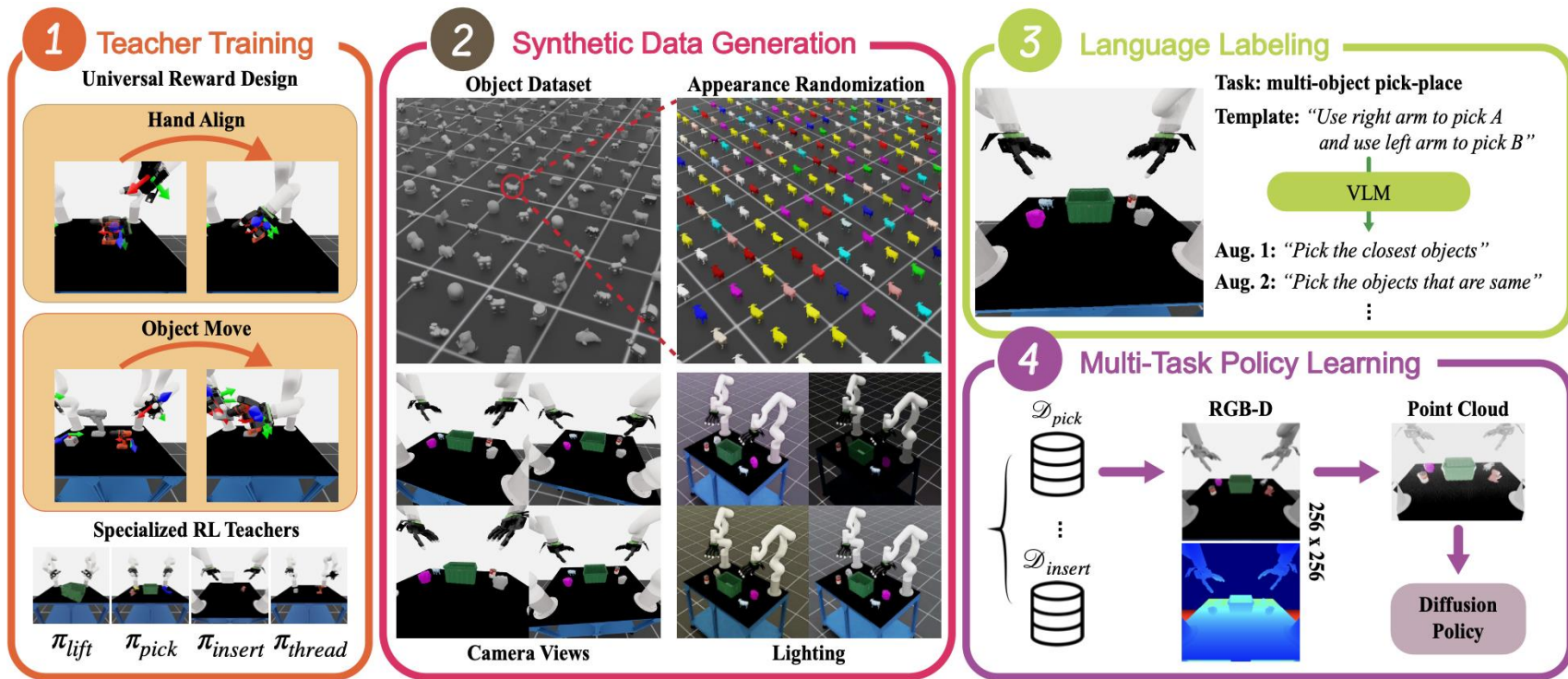
RL as a data generator for imitation

RLDC: per-skill RL teachers become a language-labeled dataset



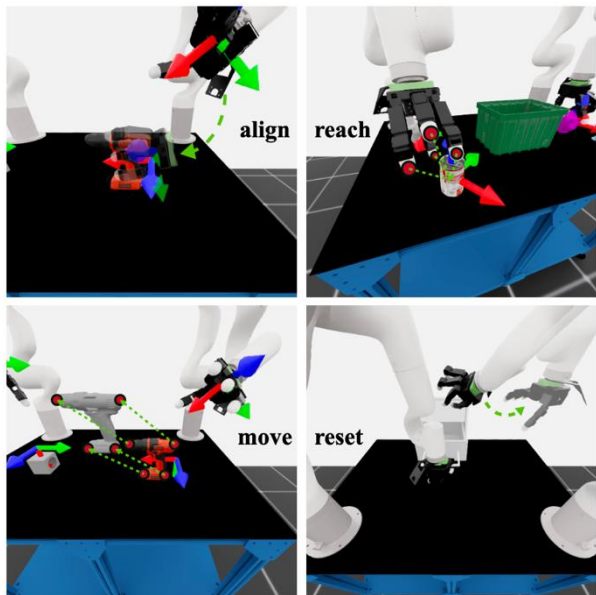
The inversion. instead of humans teleoperating data, RL specialists generate it, and a vision-language model labels it. The student is trained by imitation on the synthetic corpus.

Synthetic data-gen via massive RL



Generalized reward design

no teleoperation, and it matches hand-tuned reward design



align + contact + flexible goal + reset

Same terms solve lifting, pick-place, and drawer insertion.

~1 hour per task vs ~1 day for trajectory rewards.

0 teleoperated demos

zero-shot sim-to-real

Synthetic reward matches hand-tuning

comparable to task-specific reward design, and it beats trajectory tracking

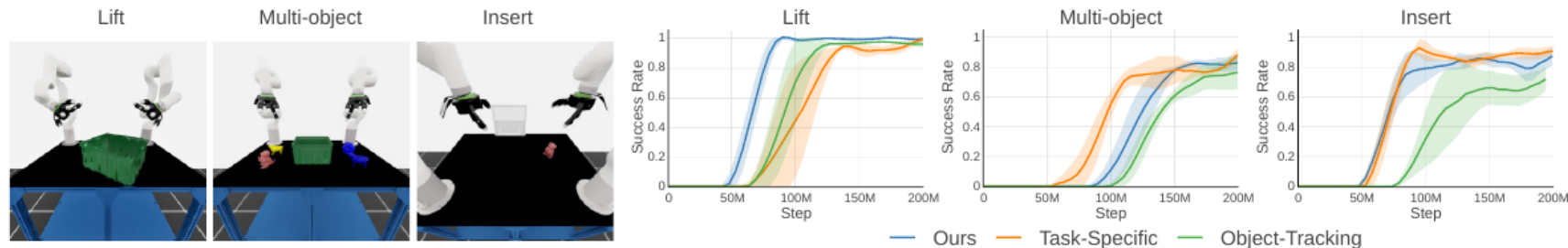


Fig. 4. (Left) Visualization of three bimanual dexterous manipulation tasks in simulation. (Right) Performance of the proposed reward design compared to baselines. Our rewards achieve performance comparable to task-specific designs and outperform the object reference trajectory baseline.

Why it works. the generalizable reward terms recover most of what an expert would hand-tune, at a fraction of the effort.

Zero-shot sim2real w/ diverse language instr.



4

PRIMITIVE 4 OF 5

Exploration and multimodal policies

a Gaussian head collapses the behaviors a robot actually needs

Why one mode is not enough

the average of two good actions is usually a bad one



$$\frac{1}{2}(a_{\text{left}} + a_{\text{right}}) \notin \mathcal{A}_{\text{good}}$$

a unimodal policy commits to the midpoint and loses both solutions

Why robots care. grasps, paths, and contacts are genuinely multimodal; collapsing them hurts robustness.

Diffusion policies, but trained online

DDiffPG: learn multimodal behavior from scratch with a policy gradient

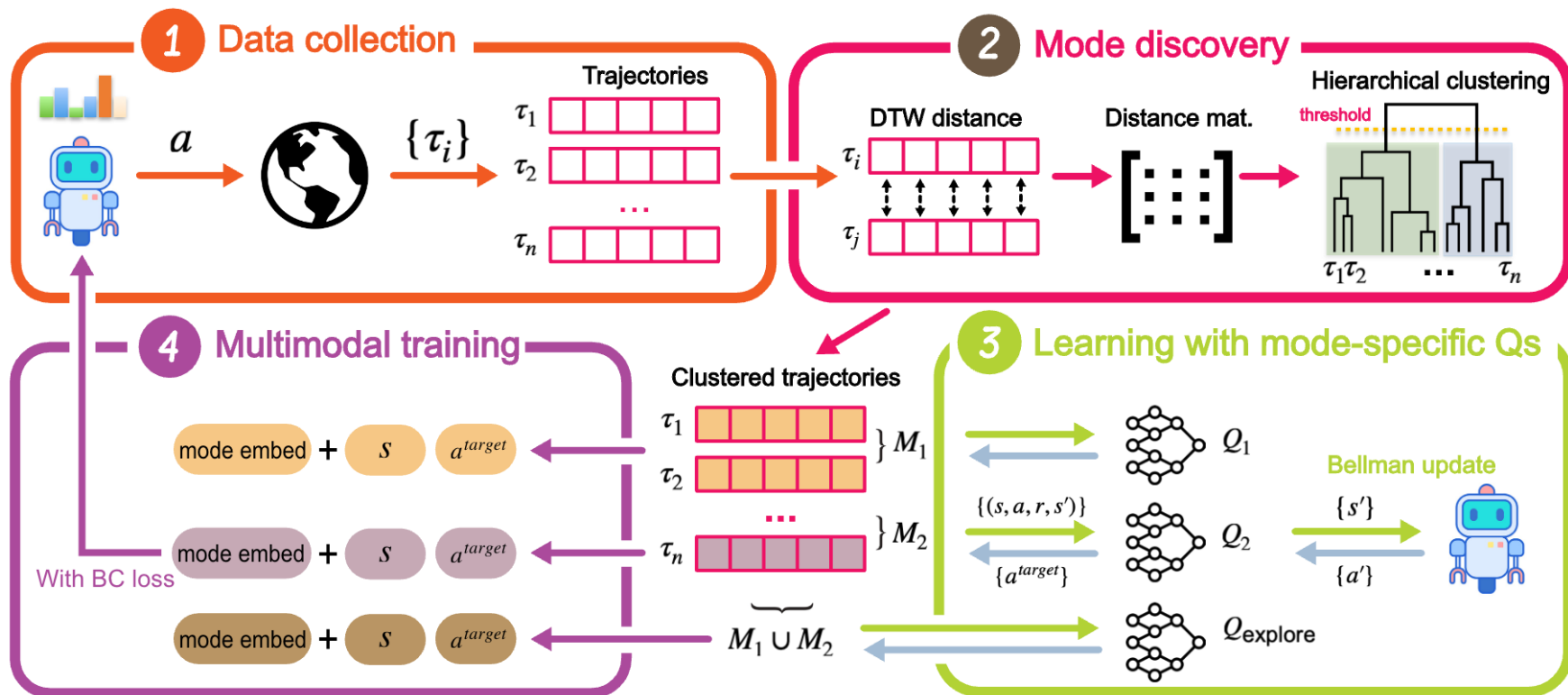


$$a_0 \sim p_{\theta}(a_0 \mid s), \quad a_k \rightarrow a_{k-1} \quad (\text{denoise})$$

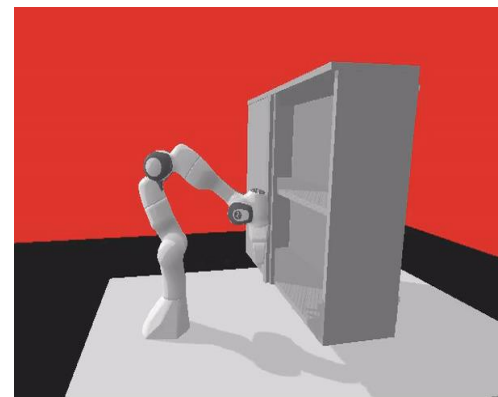
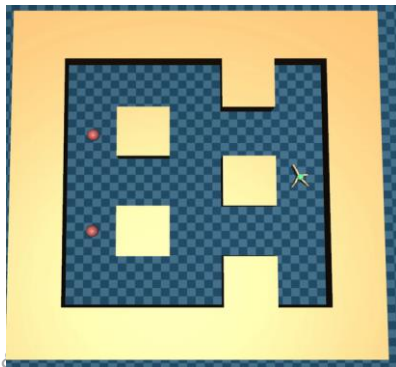
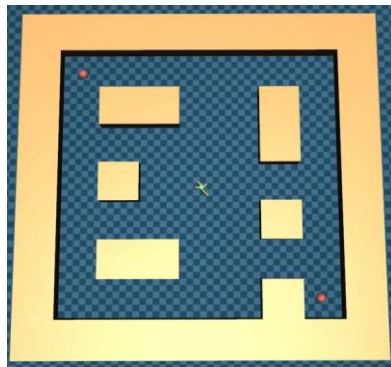
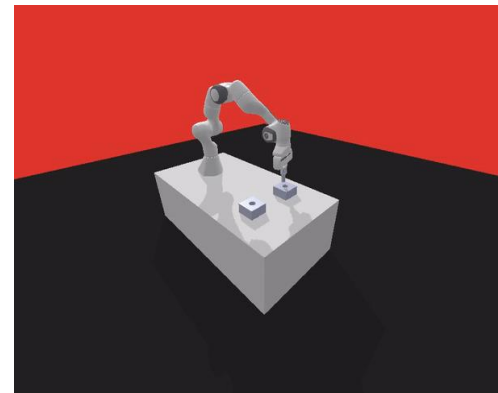
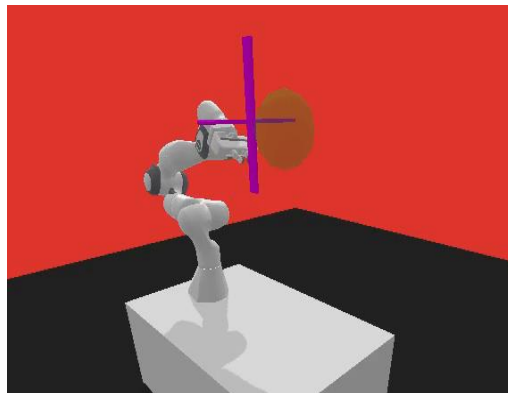
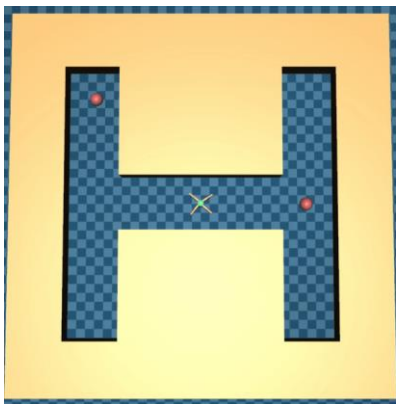
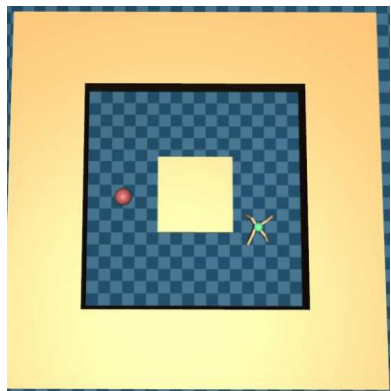
represent a multimodal distribution and still improve it with RL

Result. discovers and keeps distinct strategies where standard RL converges to one.

Learning multimodal behaviors from scratch with diffusion policy gradient



Learning multimodal behaviors from scratch with diffusion policy gradient



5

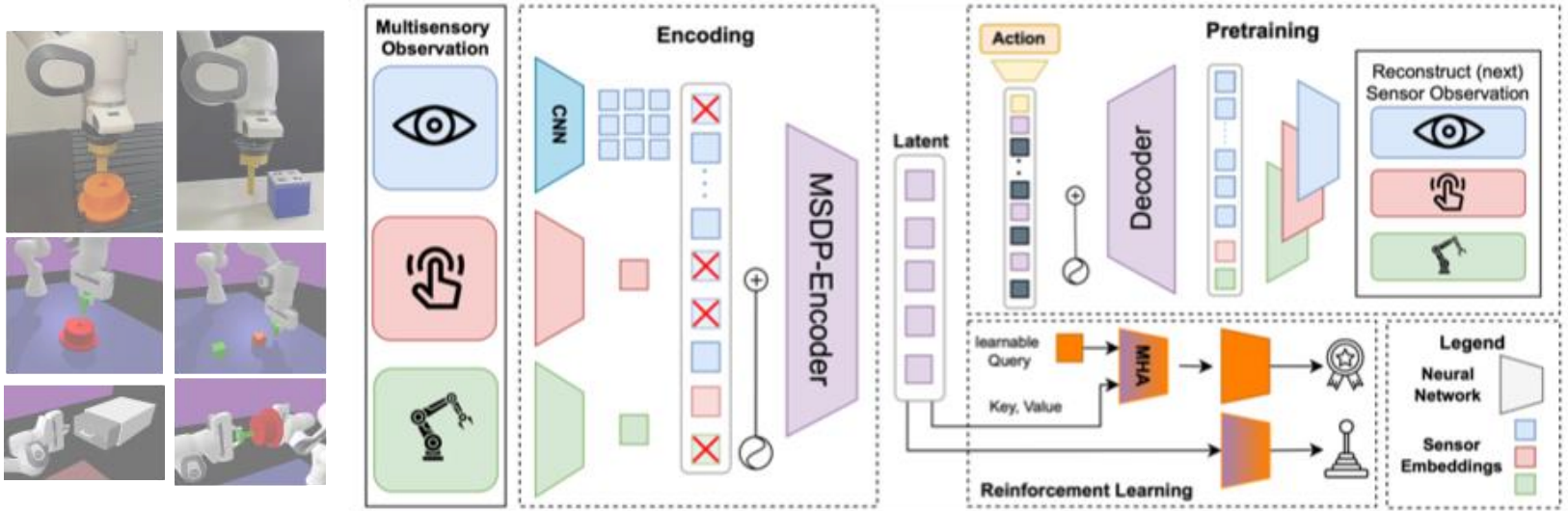
PRIMITIVE 5 OF 5

Sensing contact

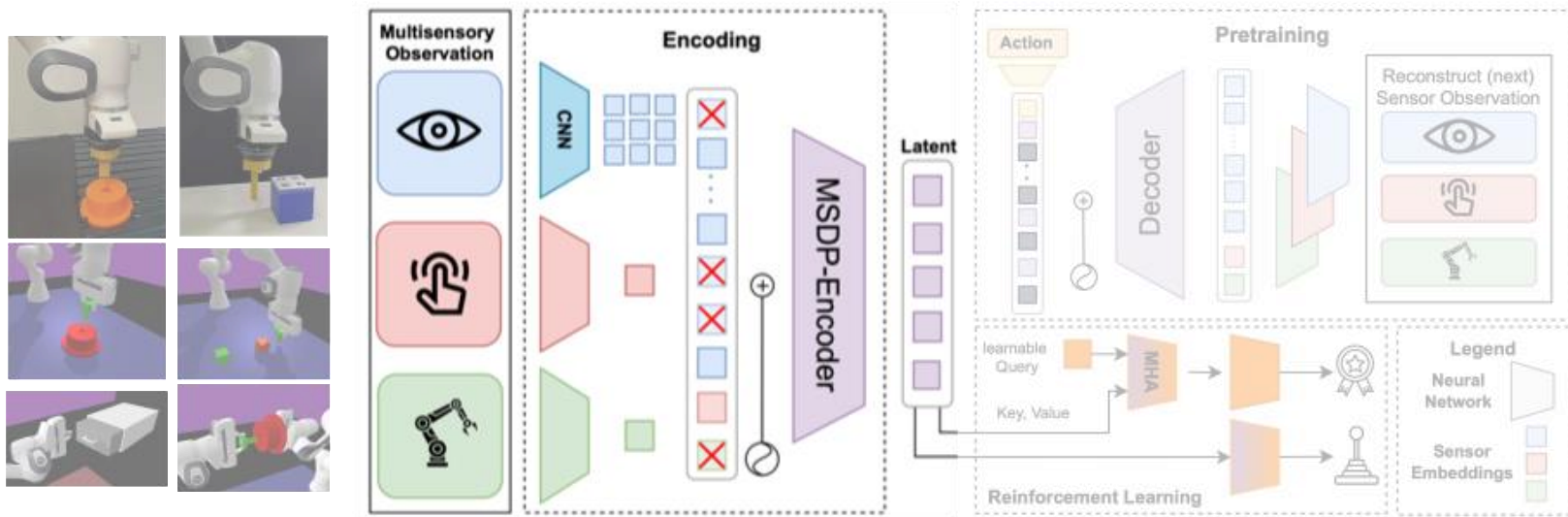
vision cannot see the interaction; the policy has to feel it

Observation is a design choice

proprioception, force-torque, and tactile, fused and pretrained

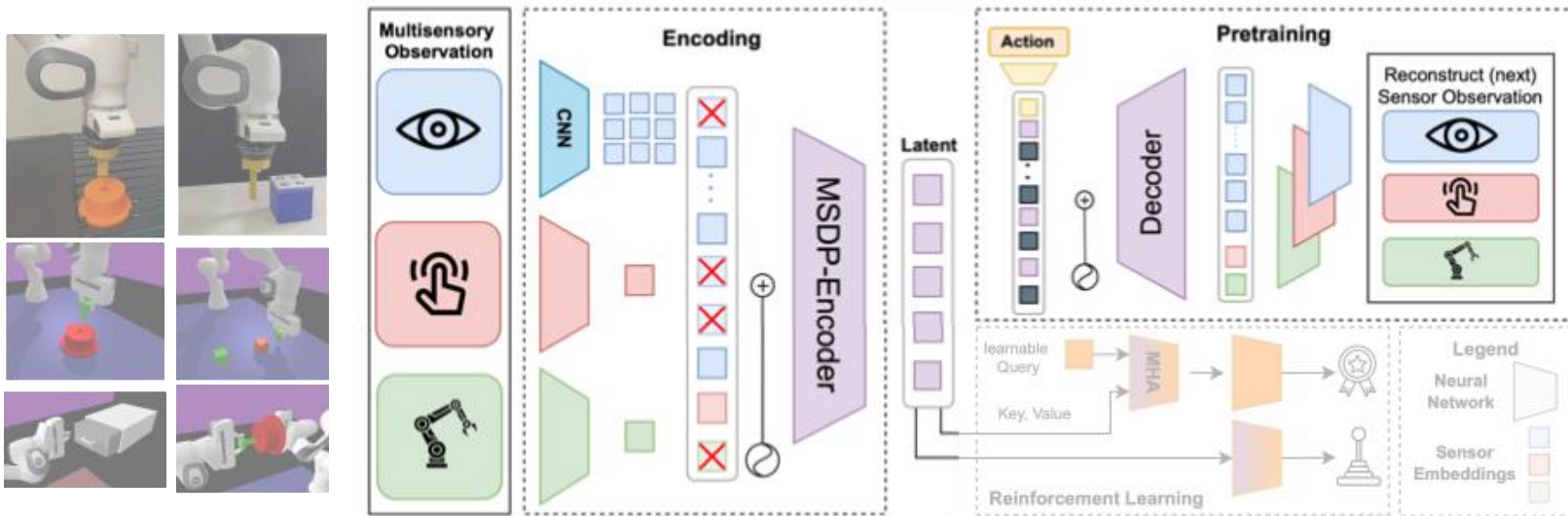


Self-Supervised Multisensory Pretraining for Contact-Rich Robot RL



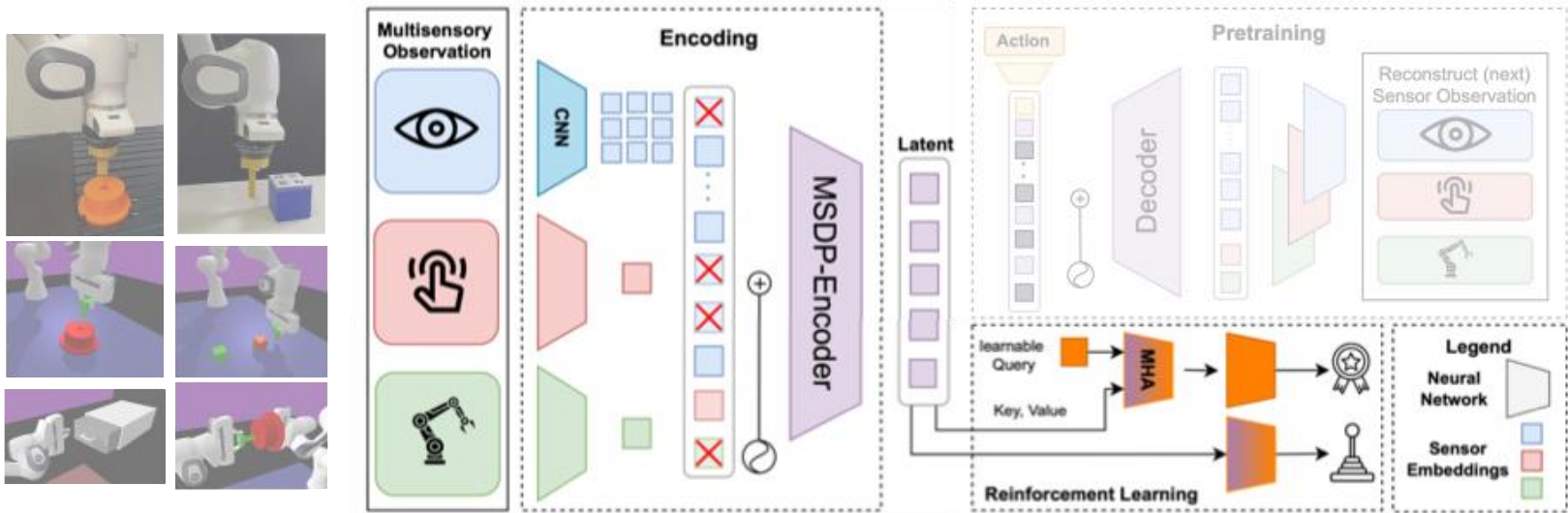
Flexible sensor fusion based on
masked autoencoding

Self-Supervised Multisensory Pretraining for Contact-Rich Robot RL



Reconstruct (next) multisensory observation from a subset of sensor embeddings.

Self-Supervised Multisensory Pretraining for Contact-Rich Robot RL



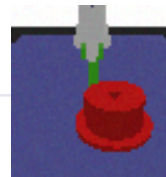
Task-specific multisensory feature extraction via cross-attention for efficient policy learning.

A contact-rich benchmark

tasks where dynamics vary and interaction must be precise

Peg Insertion

alignment under force; vision occluded near contact



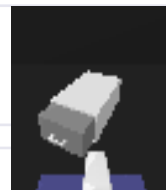
Push Cube

non-prehensile; friction and slip matter



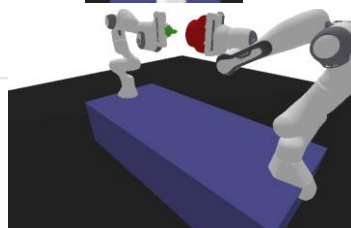
Close Drawer Gently

force-limited; reward penalizes impact



Dual-Arm Peg Insertion

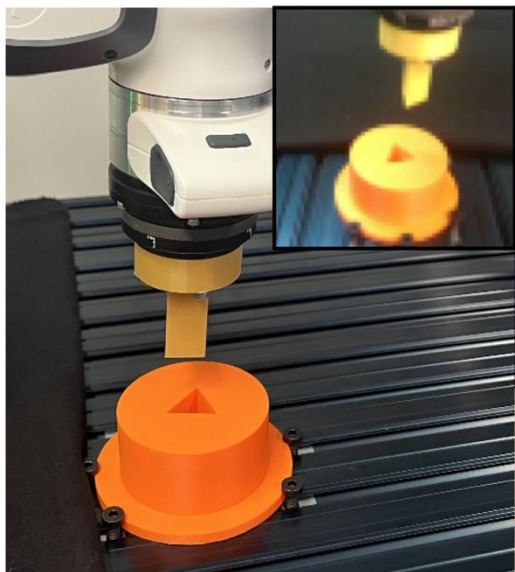
bimanual coordination, mutual contact constraints



metric: success rate vs real-robot environment steps

Peg insertion learned on the real robot

the multisensory representation makes contact-rich RL efficient

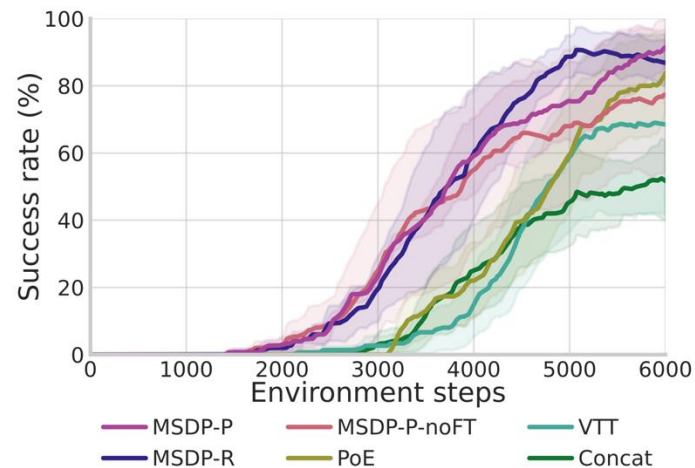


Close up & Camera View

~90% real peg-insertion success



6k env steps on hardware



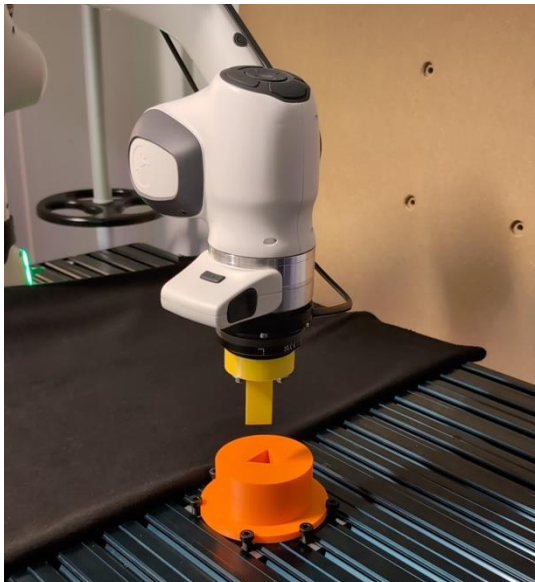
Robust where vision alone fails

high success under lighting changes, occlusion, and external forces

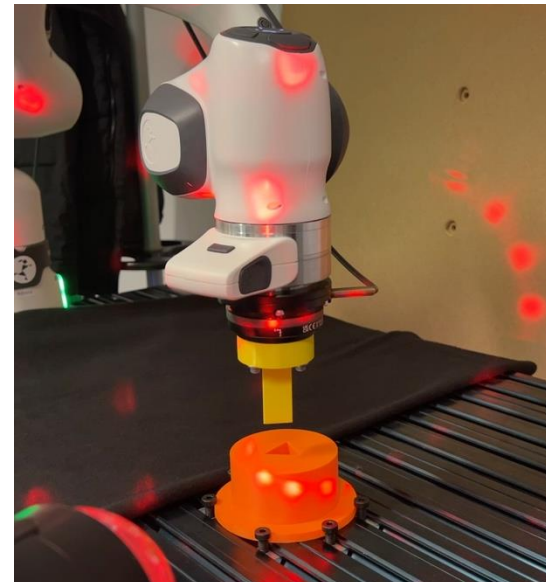


High success rate
and robust against:

Krohn et al., RA-L (to appear)



Light sources



Disco

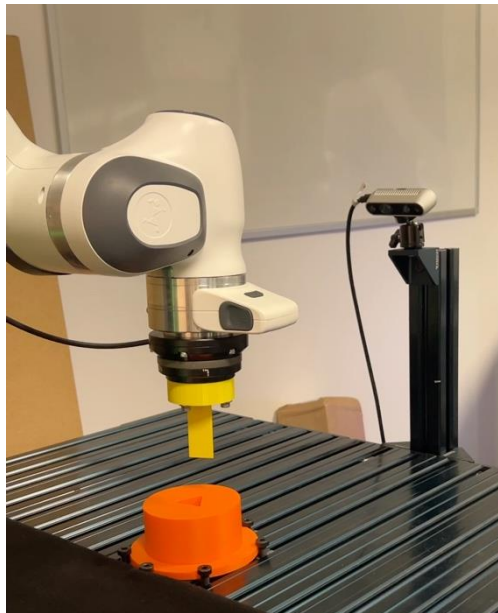
Various light conditions

Robust where vision alone fails

high success under lighting changes, occlusion, and external forces



High success rate
and robust against:

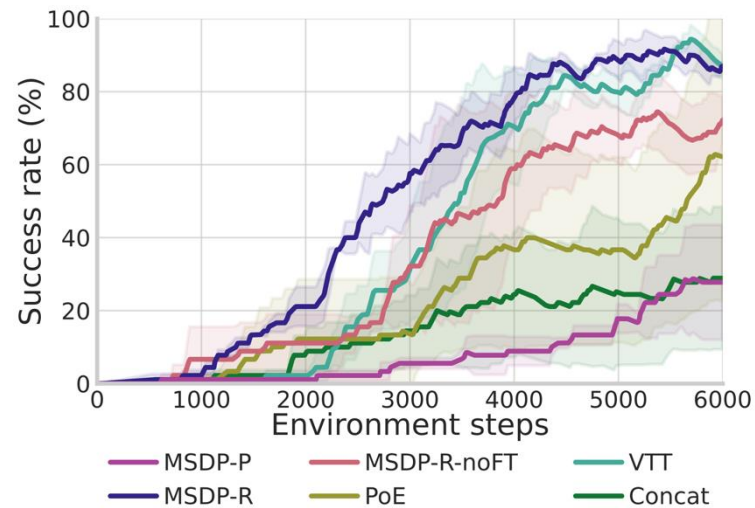
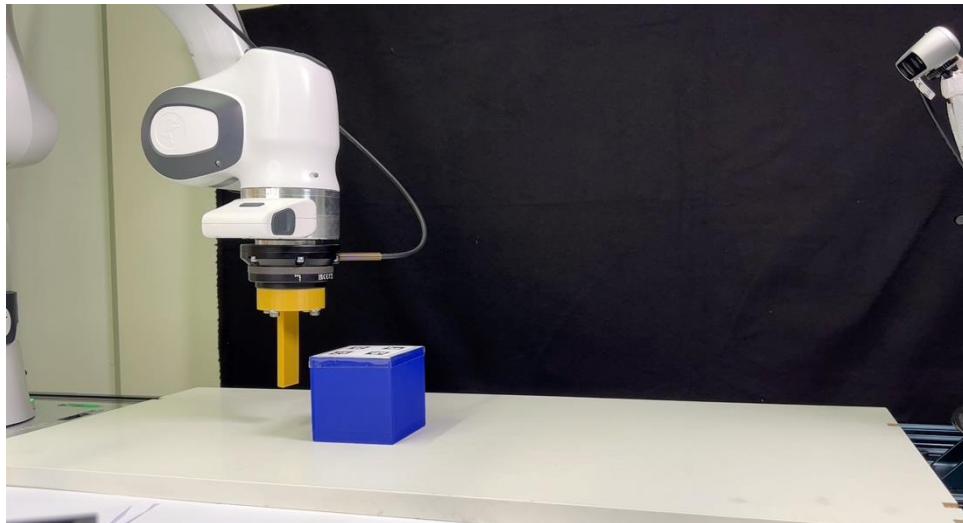


Visual Occlusions



External Forces

Push cube



Act II in one view

five primitives, five systems

Action representation

where the policy acts; correct a controller, don't replace it

BHyRL · WHOLE-MoMa

Structure

symmetry shares experience across hands

SYMDEX

Reward and data

compose physical reward terms; let RL generate the data

RLDC

Exploration

keep the modes a Gaussian would average away

DDiffPG

Sensing contact

pretrain the multisensory signal vision can't give

MSDP

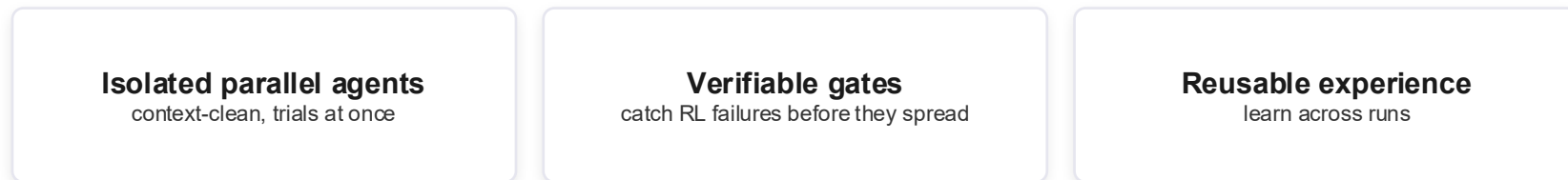


Automating the robot RL engineering loop

HARBOR: given a simulator and a task, build the whole MDP and train it



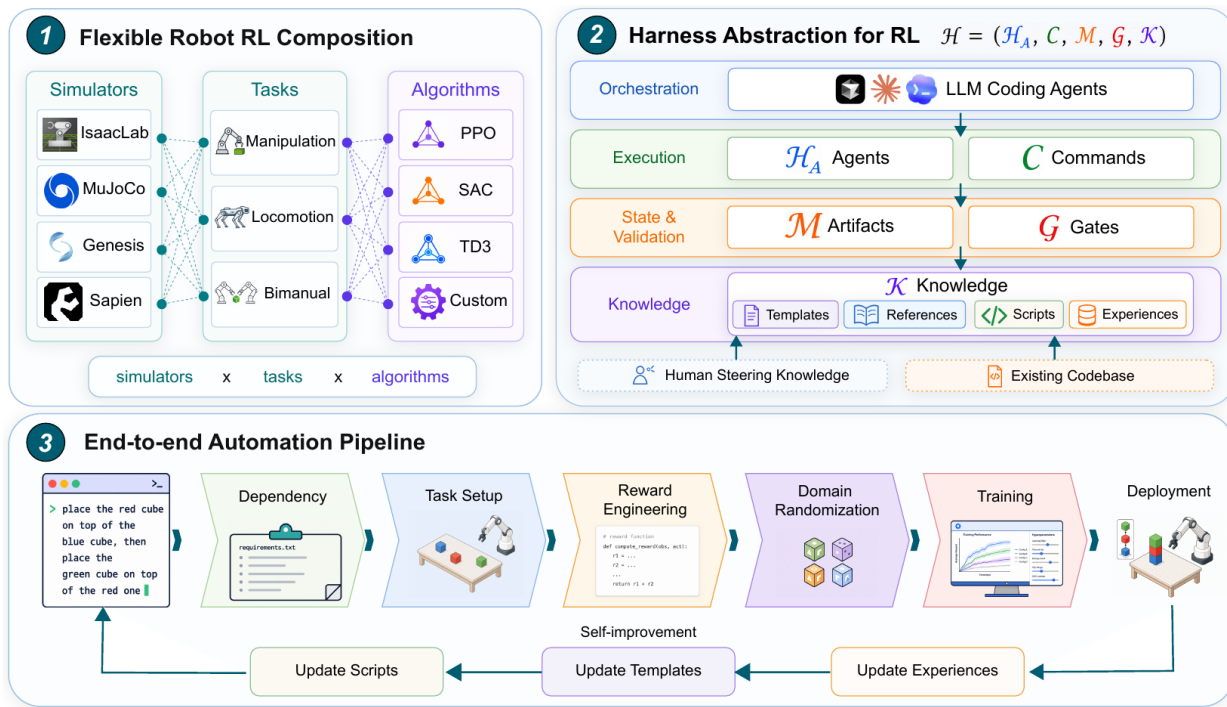
wrapped in a harness:



$$\mathcal{H}_{\text{RL}} = (\mathcal{H}_A, C, M, G, K)$$

One request, a policy that transfers

across 6 benchmarks and 16 tasks, then onto real hardware



48/50 reliable, vs 18/50 no harness

+1.3K% tuning AUC, IsaacLab

PEARL Lab, HARBOR, under review, 2026. 6 benchmarks, 16 tasks; sim-to-real on a bimanual Franka and an Allegro hand.



A c t I I I

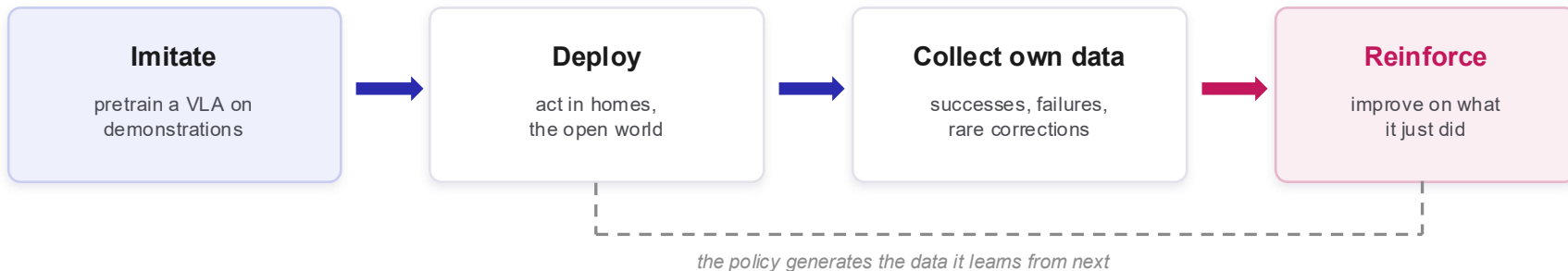
The loop that never closes

continual RL for VLAs is the open problem behind “reinforce what you imitate”



The loop never closes

a deployed robot has to keep improving from its own experience



The shift. “reinforce what you imitate” sounds like a one-time finishing step. The real regime is **continual**: the robot is never done, the data distribution moves under it, and every update has to hold up on hardware in real time.

Pour the water from teapot
Into the bowl



Close the right cabinet door



Why a robot is not an LLM with RL

the trick that made LLM post-training work does not exist here

LLM + RL

Verifiable reward. math has an answer, code has a unit test. Ground truth, cannot be faked.

Cheap, parallel rollouts. a generation is text; sample millions in a datacenter.

One ability rarely kills another. better at math does not erase writing.

VLA + RL

No verifier. “did the espresso get made” is not checkable. A learned reward gets hacked the moment you optimize it.

Every sample costs real time. seconds on hardware, a human reset, risk to the robot.

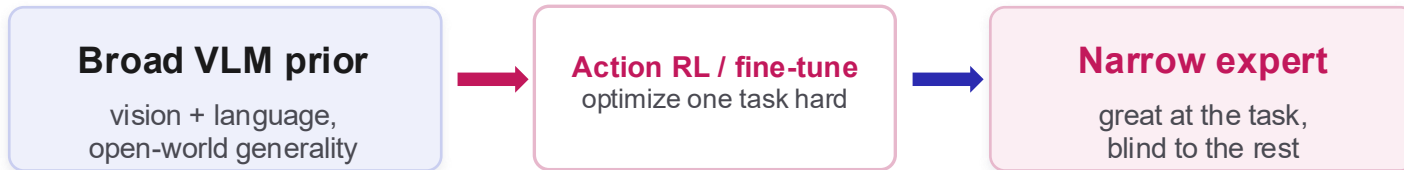
Improvement competes with generality. *the next slide — no LLM analogue.*

The reward you used for text does not transfer. a physical task has no ground-truth grader, so the reward becomes the design problem of Act II again — now under a moving distribution.

Suggested visual: split screen — a passing unit test (green check) beside a robot mid-contact with no grader. The asymmetry in one image.

Fine-tuning eats the prior it was built on

the tension with no clean answer yet



What the literature now measures

88% → ~0%

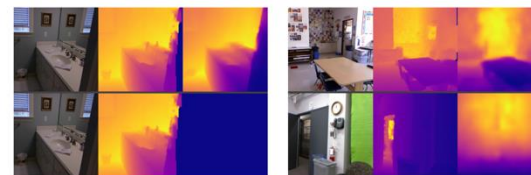
VLM object-understanding (POPE) collapses after plain action fine-tuning

DINOv2 → flat

pretrained visual features degrade to low-detail, near-uniform depth

attention drifts

fine-tuned policies fixate on irrelevant regions out of distribution



(a) DPT.

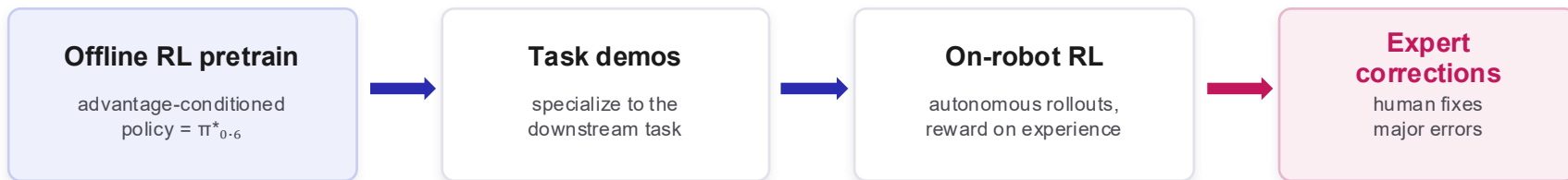
(b) Linear.

Fig. 2: Depth regression from original DINO-v2 (top-right) and OpenVLA DINO-v2 (bottom-right) using DPT head (a) and linear probing (b). Each row: input image, ground-truth, predicted depth. The original DINO-v2 estimates depths correctly, whereas OpenVLA DINO-v2 performs poorly due to catastrophic forgetting during OpenVLA training.

The crux. for an LLM, RL on one skill rarely erases the others. For a VLA the whole value *is* the broad prior — so every gain on the deployed task is paid for in lost generality. Keeping both is unsolved.

A VLA that learns from experience

$\pi^*_{0.6}$ / RECAP — the most visible “RL is back” claim for robots



What it shows. folds laundry in real homes, assembles boxes, runs an espresso station for hours. Self-improvement roughly doubles throughput over the imitation-only base.

What it admits. the loop still leans on **expert teleoperated interventions** to catch failures — the same teleoperation crutch as Able Mabel, sixty years on. That dependency is the open problem.



Where this leaves us

four open problems the next decade of robot RL runs on

1

Reward without a verifier

physical tasks have no unit test. Self-generated and VLM-defined rewards, robust to hacking under a moving distribution.

2

Improve without forgetting

let RL sharpen the deployed task while the broad vision-language prior survives. Currently unresolved.

3

Close the loop autonomously

remove the expert-correction crutch: detect own failures, reset, keep going unattended on hardware.

4

RL through chunked, flow policies

credit assignment and value learning at the action-chunk level, not the single step a Gaussian head assumes.

The honest close. “reinforce what you imitate” was the easy half. The hard half is doing it **forever, on hardware, without a verifier, and without forgetting what made the model worth deploying.**



Practical notes: sim, parallelism, compute

what to set up before you train on a robot

- Parallel sim (Isaac Gym / Isaac Lab) for on-policy methods: thousands of envs per GPU.
- Domain randomization on dynamics and appearance from day one, not as a patch later.
- Keep a fixed real-robot evaluation protocol; sim success is not the deliverable.
- Budget human time for resets explicitly; design reset-free rewards where you can.
- Log multisensory observations even if a method ignores them; you will want them later.
- Pretrain perception offline; reserve the real-robot budget for the policy.

| If you are moving from RL to robots

a checklist the milestones and the primitives agree on

- Choose the action space deliberately: task or whole-body, not the raw joint vector.
- Treat classical control as a prior to correct, not a baseline to beat.
- Exploit structure first: symmetry, priors, decomposition, before scaling data.
- Design contact-aware rewards; resist copying reference trajectories.
- Pretrain perception, especially non-visual sensing, before policy learning.
- Learn in parallel sim with randomization; keep a real-robot evaluation honest.

| Key references

the milestones and the systems behind this tutorial

SEMINAL

Kober, Bagnell & Peters, RL in Robotics: A Survey, IJRR 2013
 Levine et al., End-to-End Visuomotor Policies, JMLR 2016
 Kalashnikov et al., QT-Opt, CoRL 2018
 Andrychowicz et al., HER, NeurIPS 2017
 OpenAI et al., Dexterous In-Hand Manipulation, IJRR 2020
 Hwangbo et al., ANYmal, Science Robotics 2019
 Miki et al., Perceptive Locomotion, Science Robotics 2022
 Rudin et al., Walk in Minutes, CoRL 2022
 Kaufmann et al., Drone Racing, Nature 2023

FROM PEARL

Jauhri, Peters & Chalvatzaki, BHyRL, RA-L 2022
 Jauhri, Prasad & Chalvatzaki, WHOLE-MoMa, 2026
 Li et al., SYMDEX, CoRL 2025
 Li et al., RLDC, under review
 Li et al., DDiffPG, NeurIPS 2024
 Krohn et al., MSDP, RA-L (to appear)
 PEARL Lab, HARBOR, under review 2026